

REQUIREMENTS SPECIFICATIONS

<P06>:<ANOMALOUS LOGIN DETECTION SYSTEM VIA ELK STACK>

<TEAM MEMBER NAMES & IDS>

STUDENT ID	NAME
26100286	MOHAMMAD MUSTAFA
26100399	MUSTAFA HUSSAIN
26100015	MUHAMMAD AAFAN KHAN NIAZI
25100022	SHEHROZ FARYAD

TABLE OF CONTENTS

1.	Introduction.....	3
2.	System Actors.....	4
3.	Use Cases.....	5
3.1	Use Case Diagrams.....	5
3.2	Description of Use Cases.....	5
3.2.1	Withdraw cash.....	5
3.2.2	Transfer funds.....	6
4.	Class Diagram.....	7
4.1	Diagram.....	7
4.2	Description.....	7
5.	Sequence Diagrams.....	8
5.1	Use case Name e.g., Withdraw cash.....	8
5.2	Use case Name e.g., Transfer funds.....	8
6.	State Diagrams.....	9
6.1	Diagram details.....	9
6.2	Diagram.....	9
7.	Data Requirements.....	10
8.	Non-functional Requirements / Quality Attributes.....	11
9.	Security Requirements.....	12
10.	Security Engineer.....	13
11.	Use of Generative AI.....	14
12.	Who Did What?.....	15
13.	Review checklist.....	15

1. Introduction

<Give an overview of the project here. The overview must highlight the overall objectives of the project and its potential users and customers.>

The project is concerned with the detection of unusual or suspicious login activity in real-time using the ELK stack (Elasticsearch, Logstash, Kibana) along with Wazuh. The primary objective of the system is to collect authentication logs and search patterns that can possibly indicate a potential security problem. Such patterns are things like multiple attempts of logging in over a short span of time or abrupt alterations in the location where the user is logging in. The system is able to detect these behaviors and minimize the chances of unauthorized access.

System administrators and security engineers are the key individuals that will utilize this system. To them, the project offers a user-friendly dashboard that lets them easily view the activity of the login at a glance, and also offer instant notification whenever something out of the ordinary occurs. This implies that there is less time wasted in searching through raw logs and that action is taken much faster when an issue arises.

In general, the project will enhance the speed at which organizations can recognize and react to abnormal login activity. Having clear visibility, real time alerts and effective monitoring, system administrators and security engineers can ensure that they protect systems better and contain potential threats.

2. System Actors

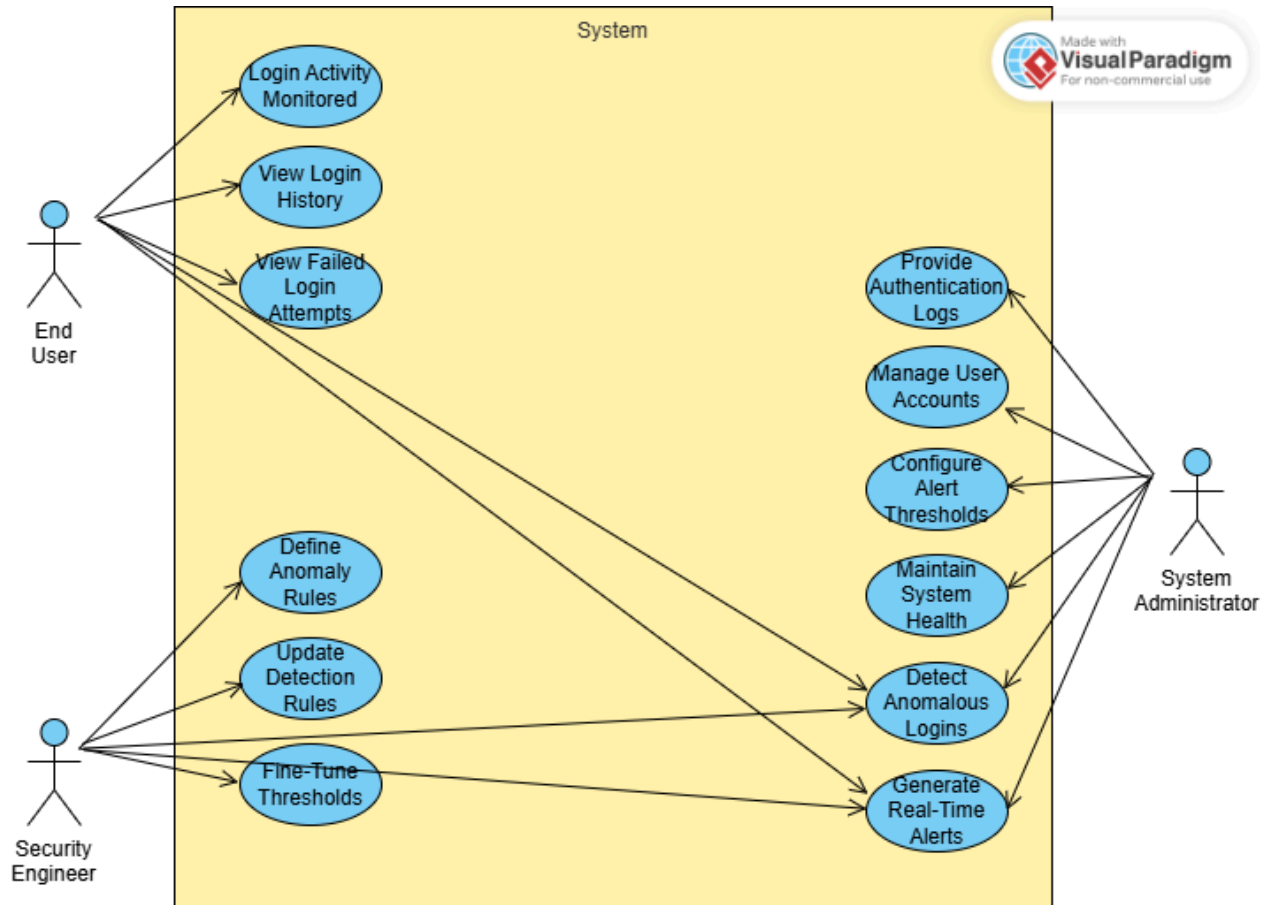
<List down the actor names and give a 2-3 lines description of the role of each actor>

Actor Name	Description
End User	The end user is the individual whose login activities are being monitored. They benefit from increased protection against suspicious or unauthorized login attempts.
Security Engineer	The security engineer defines what counts as an anomalous login and updates detection rules. They refine the system so it can adapt to new attack patterns.
System Admin	The system administrator provides authentication logs for analysis. They ensure that the necessary data is available for detecting abnormal login activities.

3. Use Cases

3.1 Use Case Diagrams

<Use standard UML notation>



3.2 Description of Use Cases

[Select 20 most important use cases of your project and create their comprehensive descriptions.]

<Write description of each use case separately using the template below.>

3.2.1 Ingest Windows Logon Events

Identifier	UC-001
Purpose	Collect Winlogon/NTLM/Kerberos events from Windows endpoints for monitoring.
Pre-conditions	Wazuh/Winlogbeat agent installed on the endpoint. TLS connectivity available to Logstash/Wazuh Manager.
Post-conditions	Events are stored in Elasticsearch under index <code>auth-windows-*</code> . SOC analysts can view and search them in Kibana.
Step #	Typical Course of Action
1.	Windows endpoint generates a logon event.
2.	The Wazuh/Winlogbeat agent collects the event.
3.	The agent securely ships the event to the Wazuh Manager/Logstash.
4.	Logstash parses and normalizes the log to ECS schema.
5.	Elasticsearch indexes the event into <code>auth-windows-*</code> .
6.	SOC analysts review data in Kibana dashboards.
7.	The use case ends.
Step #	Alternate Courses of Action
1.	If the agent is offline, logs are buffered locally. If parsing fails, the event is routed to a dead-letter index.
Step #	Exception Paths
1.	If the network is down, events are delayed until connectivity is restored.

3.2.2 Ingest Linux SSH Authentication Logs

Identifier	UC-002
Purpose	Collect and process Linux SSH authentication events for detection of anomalies.
Pre-conditions	Filebeat/Wazuh agent installed on the Linux host. Connectivity to the central collector.

Post-conditions	Events are stored in Elasticsearch under index <code>auth-linux-*</code> . SOC analysts can search and visualize them in Kibana.
Step #	Typical Course of Action
1.	A user attempts SSH login on a Linux server.
2.	The attempt is logged to <code>/var/log/auth.log</code> .
3.	Filebeat/Wazuh agent reads the log entry.
4.	The agent forwards the log to Logstash.
5.	Logstash parses and enriches the data.
6.	Elasticsearch indexes the event.
7.	Kibana dashboards display login activity.
Step #	Alternate Courses of Action
1.	If parsing fails, the event is redirected to a dead-letter index.
Step #	Exception Paths
1.	If time skew is detected, a time sync alert is triggered

3.2.3 Detect Brute Force Attempts

Identifier	UC-003
Purpose	Detect repeated failed login attempts and raise alerts for brute-force activity.
Pre-conditions	Authentication logs are being ingested from endpoints. Detection rule for brute force is active.
Post-conditions	Alert document is created in <code>alerts-security-*</code> . SOC receives a notification.
Step #	Typical Course of Action
1.	Multiple failed login attempts occur for a user or IP.
2.	Events are shipped and indexed in Elasticsearch.
3.	The detection rule runs a rolling window query.
4.	The number of failed logins exceeds the set threshold.

5.	An alert is created with details of the user/IP.
6.	SOC receives an email/webhook notification.
7.	The use case ends.
Step #	Alternate Courses of Action
1.	Whitelisted accounts are excluded from alert generation.
Step #	Exception Paths
1.	A misconfigured rule may generate false positives.

3.2.4 Detect Impossible Travel

Identifier	UC-004
Purpose	Identify successful logins from distant geolocations within an infeasible time window.
Pre-conditions	GeoIP enrichment enabled on login events. Historical login data for users available.
Post-conditions	Alert is raised for suspicious travel patterns. SOC is notified for investigation.
Step #	Typical Course of Action
1.	A user logs in from Location A.
2.	Shortly after, the same user logs in from Location B.
3.	Detection engine calculates distance and time difference.
4.	If travel speed exceeds threshold, the rule triggers an alert.
5.	Alert document is created in Elasticsearch.
6.	SOC analyst reviews the case.
7.	The use case ends.
Step #	Alternate Courses of Action
1.	VPN IP ranges are excluded from impossible travel checks.

Step #	Exception Paths
1.	If geo data is missing, the calculation cannot be performed.

3.2.5 Off-Hours Login Detection

Identifier	UC-005
Purpose	Detect user logins outside of their normal working hours.
Pre-conditions	Historical baseline of login times established. Detection rule for off-hours logins enabled.
Post-conditions	An alert is raised for anomalous login times. SOC analyst investigates the alert.
Step #	Typical Course of Action
1.	A user logs in at an unusual time.
2.	Event is ingested and enriched with timestamp metadata.
3.	The detection rule compares login time against user's baseline.
4.	A deviation outside the normal working window is detected.
5.	Alert is created in Elasticsearch.
6.	SOC analyst reviews and validates the alert
7.	The use case ends.
Step #	Alternate Courses of Action
1.	Known maintenance or travel accounts may be whitelisted.
Step #	Exception Paths
1.	Lack of baseline data prevents evaluation of off-hours logins.

3.2.6 New Device / User-Agent Anomaly

Identifier	UC-006
Purpose	Detect logins from a device or browser not previously seen for the account.

Pre-conditions	Login events are being ingested into Elasticsearch. User profile history available.
Post-conditions	Alert document is created in alerts-security-*. SOC notified for investigation.
Step #	Typical Course of Action
1.	User logs in from a device or browser not previously seen.
2.	Event is ingested into Elasticsearch.
3.	Detection engine compares device/user-agent against profile history.
4.	Rule identifies it as first-seen for the user.
5.	Alert is created in Elasticsearch.
6.	SOC analyst reviews and validates the alert
7.	The use case ends.
Step #	Alternate Courses of Action
1.	Known corporate devices are whitelisted from alerts.
Step #	Exception Paths
1.	Missing user-agent string prevents comparison.

3.2.7 Privileged Account Watch

Identifier	UC-007
Purpose	Apply stricter monitoring and thresholds for privileged accounts.
Pre-conditions	Privileged accounts identified and tagged. Detection rules active.
Post-conditions	Alerts raised on anomalies for privileged accounts. SOC notified for urgent triage.
Step #	Typical Course of Action
1.	Privileged account logs in.
2.	Event ingested and tagged as privileged.
3.	Detection engine applies stricter thresholds.

4.	Any anomaly triggers alerts immediately.
5.	SOC notified for urgent triage.
6.	SOC analyst reviews and validates the alert
7.	The use case ends.
Step #	Alternate Courses of Action
1.	Service accounts explicitly tagged safe may be excluded.
Step #	Exception Paths
1.	Privileged tagging misconfigured – account treated as normal.

3.2.8 Triage Alert in Kibana

Identifier	UC-009
Purpose	Enable SOC analyst to investigate and triage alerts in Kibana.
Pre-conditions	Alerts available in Kibana dashboard.
Post-conditions	Alert status updated (Open/Acknowledged/Closed). Notes added to the case.
Step #	Typical Course of Action
1.	SOC analyst receives alert notification.
2.	Analyst opens the alert in Kibana.
3.	Analyst checks related failures and IP reputation.
4.	Analyst pivots to dashboards for activity timeline.
5.	Analyst sets alert status.
6.	Notes are added to the case.
7.	The use case ends.
Step #	Alternate Courses of Action
1.	Analyst escalates directly to IR team without closing alert.
Step #	Exception Paths

1.	Kibana dashboard unavailable – triage delayed.
----	--

3.2.9 Contain Account

Identifier	UC-009
Purpose	Contain compromised accounts through disable/reset actions.
Pre-conditions	Compromise confirmed by SOC analyst.
Post-conditions	Account disabled or password reset. Tokens revoked, logs updated.
Step #	Typical Course of Action
1.	SOC confirms account compromise.
2.	Incident responder creates containment ticket.
3.	Account disabled or password reset forced.
4.	Active tokens/sessions revoked.
5.	Compromised device isolated.
6.	Incident log updated.
7.	The use case ends.
Step #	Alternate Courses of Action
1.	If compromise only suspected, account monitored more closely.
Step #	Exception Paths
1.	Critical service account cannot be disabled immediately.

3.2.10 Tune Detection

Identifier	UC-010
Purpose	Refine detection rules to reduce false positives.
Pre-conditions	Alerts reviewed and validated as false positives.

Post-conditions	Updated rule deployed to production. Improved alert accuracy.
Step #	Typical Course of Action
1.	Security engineer reviews false positives.
2.	Engineer adjusts thresholds/exclusions.
3.	Rule tested on historical data.
4.	Updated rule promoted to production.
5.	Alerts reduced while accuracy maintained.
6.	The use case ends.
Step #	Alternate Courses of Action
1.	Engineer reverts changes if results worsen.
Step #	Exception Paths
1.	Misconfiguration disables rule entirely.

3.2.11 Generate Weekly Report

Identifier	UC-011
Purpose	Automate weekly reporting of login anomalies.
Pre-conditions	Scheduler and reporting job configured.
Post-conditions	Weekly report generated and distributed.
Step #	Typical Course of Action
1.	Scheduler triggers weekly job.
2.	Elasticsearch queries summarize key metrics.
3.	Results formatted into PDF/CSV.
4.	Report emailed to stakeholders.
5.	The use case ends.
6.	

7.	
Step #	Alternate Courses of Action
1.	Report exported manually if automation fails.
Step #	Exception Paths
1.	Report job fails due to missing data.

3.2.12 Monitor VPN Logins

Identifier	UC-013
Purpose	Detect anomalies in VPN login attempts.
Pre-conditions	VPN logs forwarded to ELK.
Post-conditions	Alerts generated for suspicious VPN activity.
Step #	Typical Course of Action
1.	User attempts VPN login.
2.	VPN server generates authentication event.
3.	Event ingested into Elasticsearch.
4.	Detection checks for unusual patterns.
5.	Alert generated if anomaly found.
6.	The use case ends.
7.	
Step #	Alternate Courses of Action
1.	None
Step #	Exception Paths
1.	None

3.2.13 Detect MFA Failures

Identifier	UC-013
Purpose	Alert on repeated failed MFA attempts.
Pre-conditions	MFA events ingested into Elasticsearch.
Post-conditions	Alerts created for excessive MFA failures.
Step #	Typical Course of Action
1.	User attempts login with MFA.
2.	MFA challenge fails repeatedly.
3.	Events ingested and correlated.
4.	Rule detects failures exceed threshold.
5.	Alert is created.
6.	The use case ends.
Step #	Alternate Courses of Action
1.	None
Step #	Exception Paths
1.	None

3.2.14 Detect MFA Failures

Identifier	UC-014
Purpose	Collect login events from AzureAD/Okta.
Pre-conditions	Cloud IdP connector/API configured.
Post-conditions	Cloud login events available in ELK for analysis.
Step #	Typical Course of Action

1.	Cloud IdP generates login event.
2.	Connector/API pulls logs into Logstash.
3.	Events normalized to ECS.
4.	Elasticsearch indexes events.
5.	SOC views sign-ins in Kibana.
6.	The use case ends.
Step #	Alternate Courses of Action
1.	User retries and eventually succeeds within normal limits → event logged but no alert.
Step #	Exception Paths
1.	MFA provider outage causes false spikes in failures.
2.	Logs missing challenge results prevent accurate detection.

3.2.15 Detect SSO Anomalies

Identifier	UC-015
Purpose	Identify unusual SSO activity such as assertion mismatches or unexpected issuers.
Pre-conditions	SSO events ingested and enriched with metadata.
Post-conditions	Alerts generated for suspicious SSO events. SOC notified for review.
Step #	Typical Course of Action
1.	User authenticates via SSO.
2.	Assertion metadata compared against expected values.
3.	Detection rule identifies anomaly (issuer mismatch, replay attempt, unusual audience).
4.	Alert generated and sent to SOC.
6.	The use case ends.
Step #	Alternate Courses of Action

1.	If anomaly is due to a legitimate new SSO integration, engineer updates trusted metadata list.
Step #	Exception Paths
1.	Logs missing SAML/OIDC fields prevent anomaly detection.

3.2.16 Monitor API Access Keys

Identifier	UC-016
Purpose	Detect suspicious usage of API keys (e.g., unusual source IP or volume).
Pre-conditions	API access logs ingested.
Post-conditions	Alerts raised on anomalous API key behavior.
Step #	Typical Course of Action
1.	API key is used for authentication.
2.	Event is logged and ingested.
3.	Detection checks for unusual IP, region, or request volume.
4.	Anomaly detected triggers alert.
6.	SOC notified to investigate.
7.	The use case ends.
Step #	Alternate Courses of Action
1.	If key is used from known new environment, SOC may whitelist after validation.
Step #	Exception Paths
1.	API logging disabled or incomplete prevents anomaly detection.

3.2.17 Detect Login from Blocked Countries

Identifier	UC-017
Purpose	Alert on logins originating from restricted or sanctioned countries.
Pre-conditions	GeoIP enrichment enabled on login events. Restricted country list maintained.
Post-conditions	Alerts raised for logins from blocked geographies.
Step #	Typical Course of Action
1.	User attempts login from a blocked country.
2.	GeoIP processor enriches the event with location.
3.	Detection compares against restricted list.
4.	Alert generated and sent to SOC.
6.	SOC notified to investigate.
7.	The use case ends.
Step #	Alternate Courses of Action
1.	If login is from a traveling executive with exception approval, SOC closes alert after validation.
Step #	Exception Paths
1.	IP mis-geolocation triggers false positive alert.

3.2.18 Detect Shared Account Usage

Identifier	UC-018
Purpose	Detect concurrent use of the same account by multiple distinct users/devices.
Pre-conditions	Concurrent session monitoring enabled.
Post-conditions	Alerts raised for shared account suspicion.
Step #	Typical Course of Action
1.	Multiple users log in with the same account.
2.	System detects concurrent sessions from different devices/IPs.
3.	Rule flags account as possibly shared.
4.	Alert generated and sent to SOC.
5.	The use case ends.
Step #	Alternate Courses of Action
1.	If account is a legitimate shared service account with exceptions, SOC closes alert.
Step #	Exception Paths
1.	NATed corporate IP addresses make different users appear as one.

3.2.19 Monitor Administrative Console Logins

Identifier	UC-019
Purpose	Monitor and scrutinize logins into sensitive administrative consoles.
Pre-conditions	Admin console logs ingested.
Post-conditions	Alerts raised for anomalous admin console activity.
Step #	Typical Course of Action
1.	Admin console login attempt occurs.
2.	Event ingested and tagged as “high-value”.
3.	Detection rule applies stricter anomaly checks (off-hours, new IP, failed attempts).
4.	Alert raised if anomaly detected.
5.	The use case ends.
Step #	Alternate Courses of Action
1.	If login is part of scheduled maintenance, alert is acknowledged and closed quickly.
Step #	Exception Paths
1.	Logging misconfiguration causes missed or incomplete admin console events.

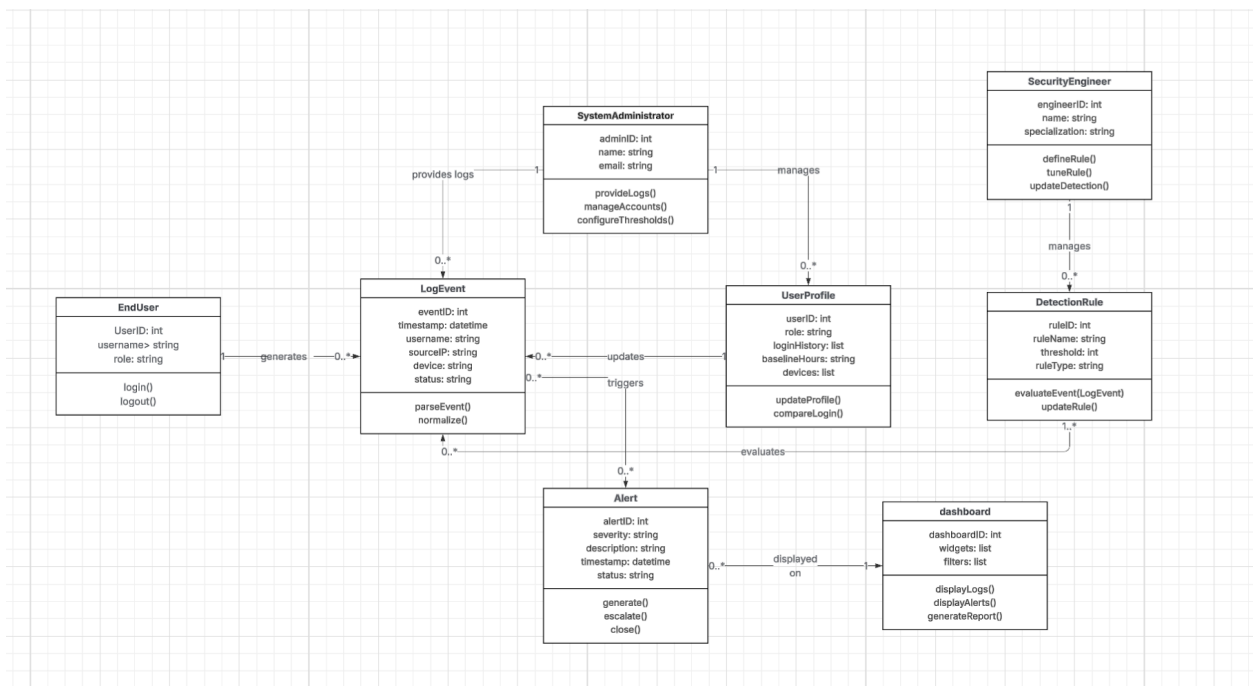
3.2.20 Detect Password Spray Attack

Identifier	UC-020
Purpose	Detect attempts where an attacker tries a common password across many different accounts.
Pre-conditions	Authentication logs from multiple accounts are ingested. Detection rules for password spraying are enabled.
Post-conditions	Alert generated when a threshold of failed logins from one IP against many accounts is exceeded.
Step #	Typical Course of Action
1.	Attacker attempts login against multiple accounts using the same password.
2.	Authentication logs are ingested into Elasticsearch.
3.	Detection engine analyzes failed logins by source IP across accounts.
4.	Threshold exceeded (e.g., >10 accounts in 5 minutes).
5.	Alert created and stored in alerts index. SOC involved.
Step #	Alternate Courses of Action
1.	If activity is from a legitimate penetration test, SOC acknowledges and closes the alert
Step #	Exception Paths
1.	Shared proxy or VPN IP causes multiple user logins to appear suspicious.

4. Class Diagram

4.1 Diagram

<Use standard UML notation>



4.2 Description

<Give brief description/purpose of each class in the class diagram. Give readable names to classes, attributes and operations.>

1. LogEvent:

Represents a single authentication or login attempt from any source (Windows, Linux, VPN, MFA, SSO, API).

- **Attributes:** eventID, timestamp, username, sourceIP, device, status.
- **Operations:** parseEvent(), normalize()

- **Purpose:** Provides a standardized format for authentication events ingested into the system.

2. UserProfile:

Maintains historical login behavior for each user to help detect anomalies.

- **Attributes:** userID, role, loginHistory, baselineHours, devices
- **Operations:** updateProfile(), compareLogin()
- **Purpose:** Stores baselines (normal login times, locations, devices) and supports anomaly detection.

3. DetectionRule:

Encapsulates detection logic for identifying anomalous logins (brute force, impossible travel, password spray, etc.).

- **Attributes:** ruleID, ruleName, threshold, ruleType
- **Operations:** evaluateEvent(LogEvent), updateRule()
- **Purpose:** Defines, applies, and manages anomaly detection criteria.

4. Alert:

Represents an anomaly detected by the system that requires review.

- **Attributes:** alertID, severity, description, timestamp, status
- **Operations:** generate(), escalate(), close()
- **Purpose:** Provides structured information about detected anomalies and tracks their resolution status.

5. Dashboard:

Provides visualization and reporting for logs, anomalies, and alerts.

- **Attributes:** dashboardID, widgets, filters
- **Operations:** displayLogs(), displayAlerts(), generateReport()
- **Purpose:** Allows administrators and security staff to review events, investigate anomalies, and export reports.

6. SystemAdministrator:

Actor class representing administrators who manage system configurations.

- **Attributes:** adminID, name, email
- **Operations:** provideLogs(), manageAccounts(), configureThresholds()
- **Purpose:** Ensures logs are ingested, thresholds are configured, and system health is maintained.

7. SecurityEngineer:

Actor class representing engineers who design and refine detection logic.

- **Attributes:** engineerID, name, specialization
- **Operations:** defineRule(), tuneRule(), updateDetection()
- **Purpose:** Manages the continuous improvement of anomaly detection accuracy and rules.

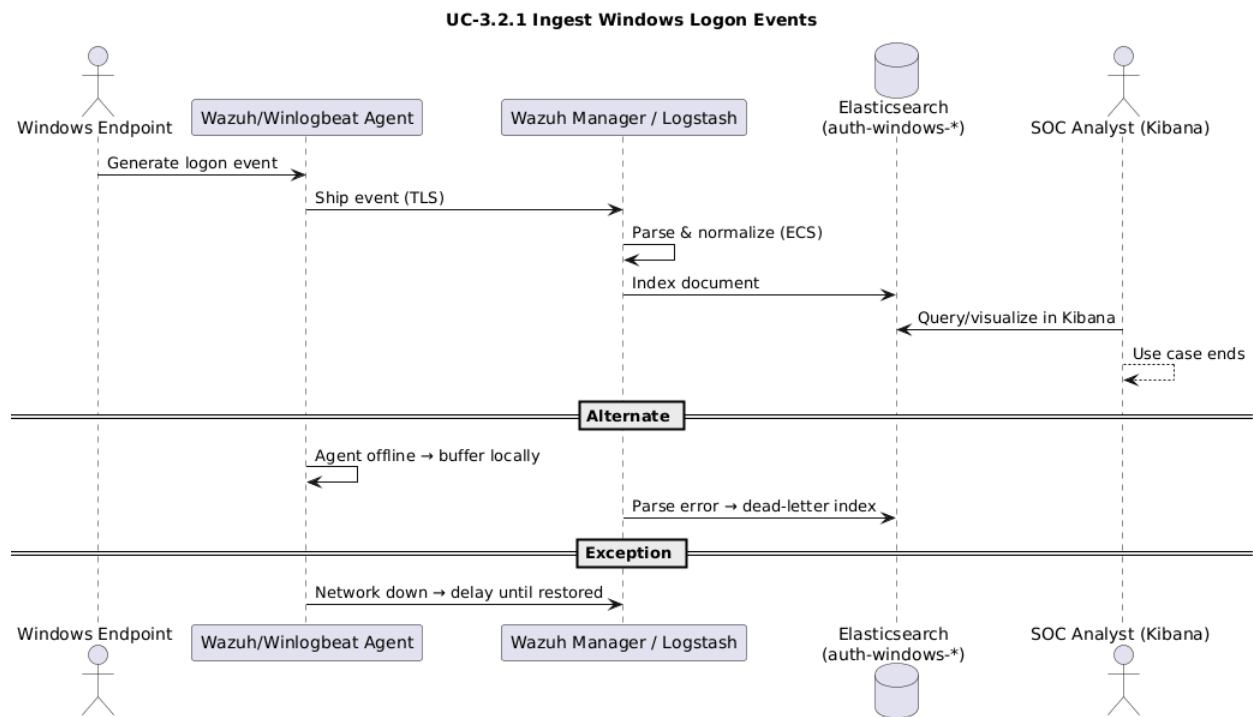
8. EndUser:

Represents individuals whose authentication activities are being monitored.

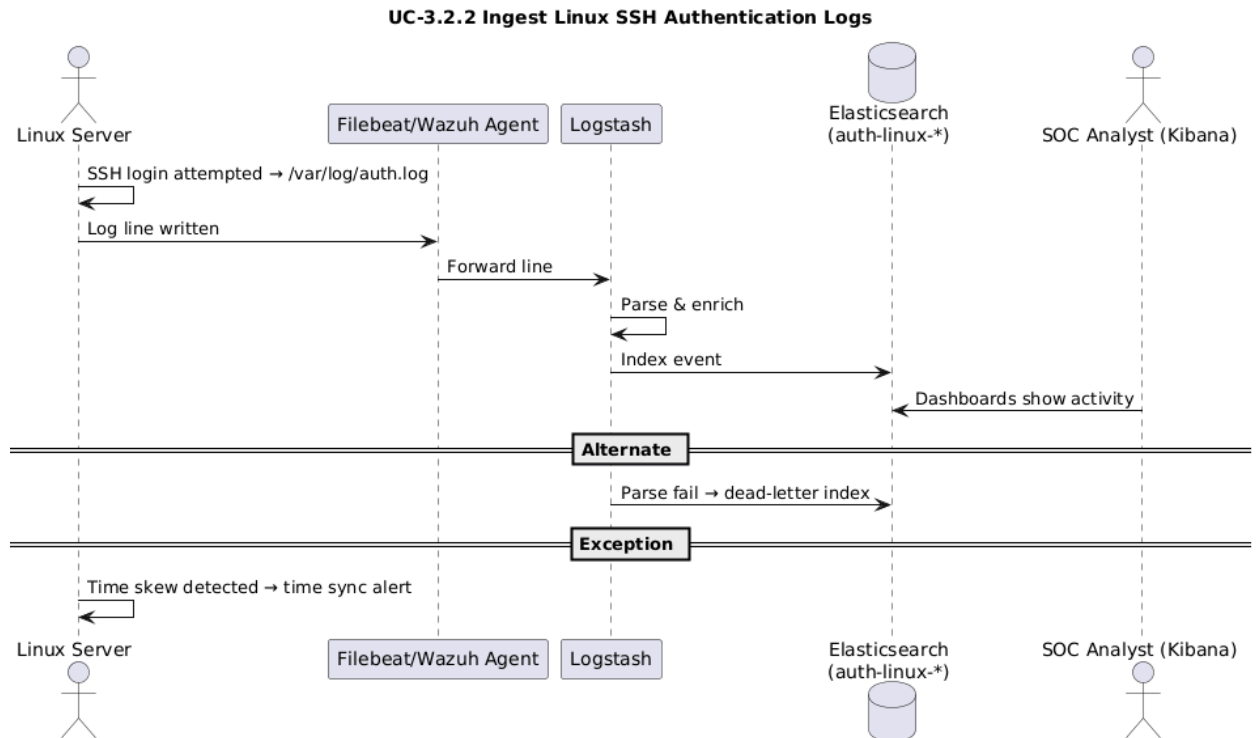
- **Attributes:** userID, username, role
- **Operations:** login(), logout()
- **Purpose:** Generates authentication events that are monitored for suspicious behavior.

5. Sequence Diagrams

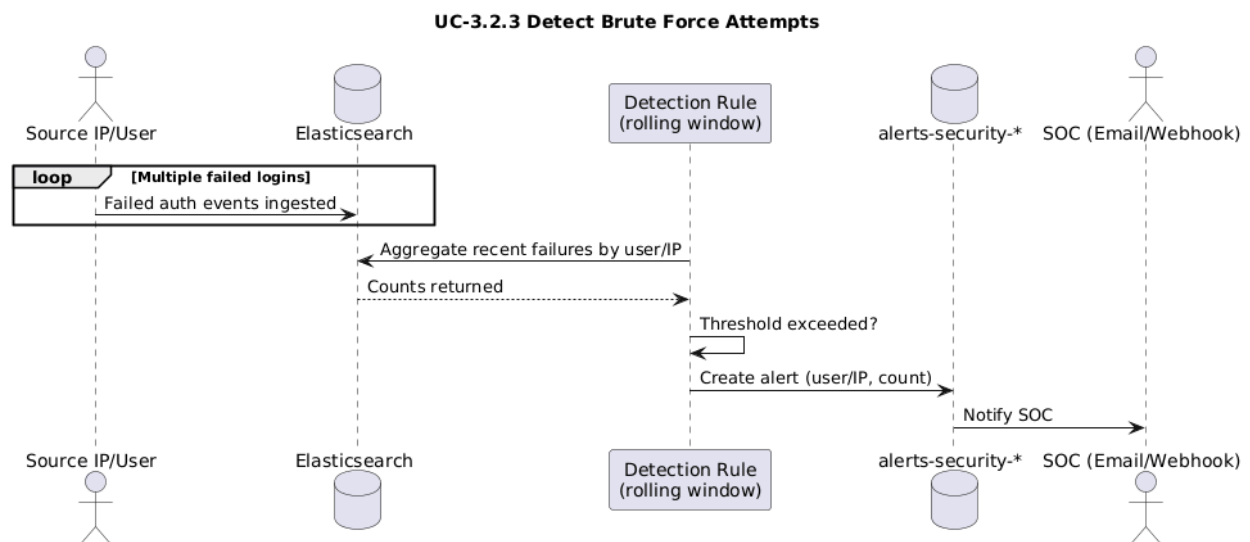
5.1 Ingest Windows Logon Events



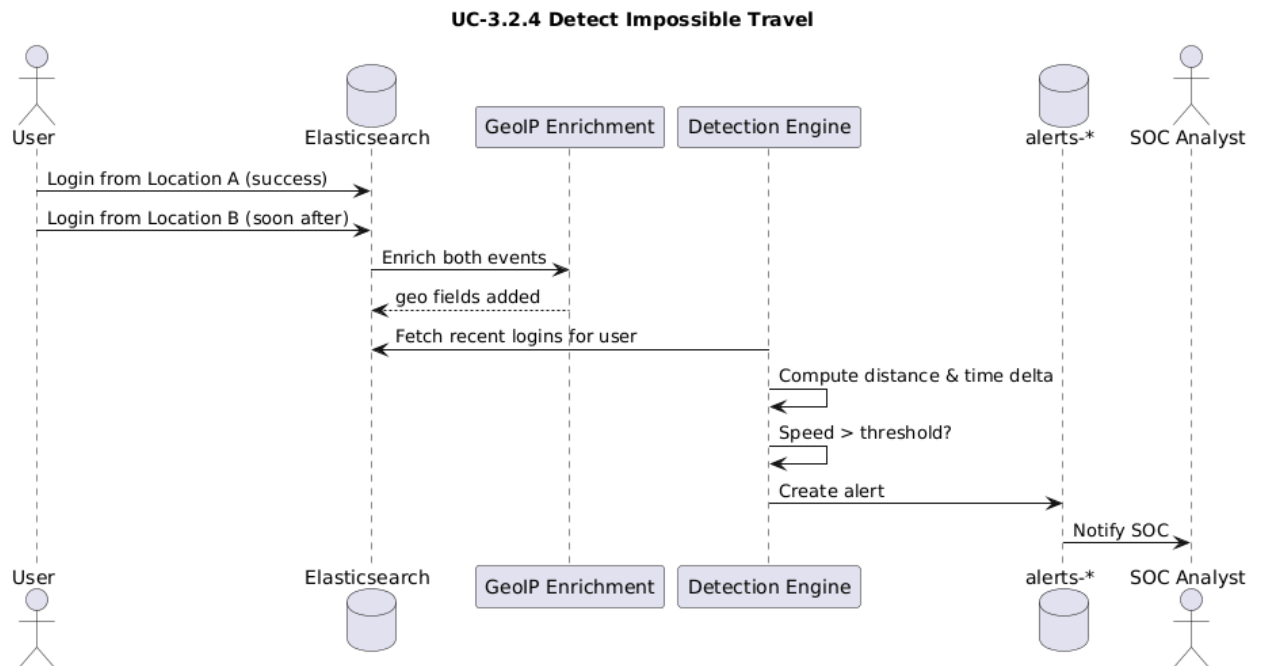
5.2 Ingest Linux SSH Authentication Logs



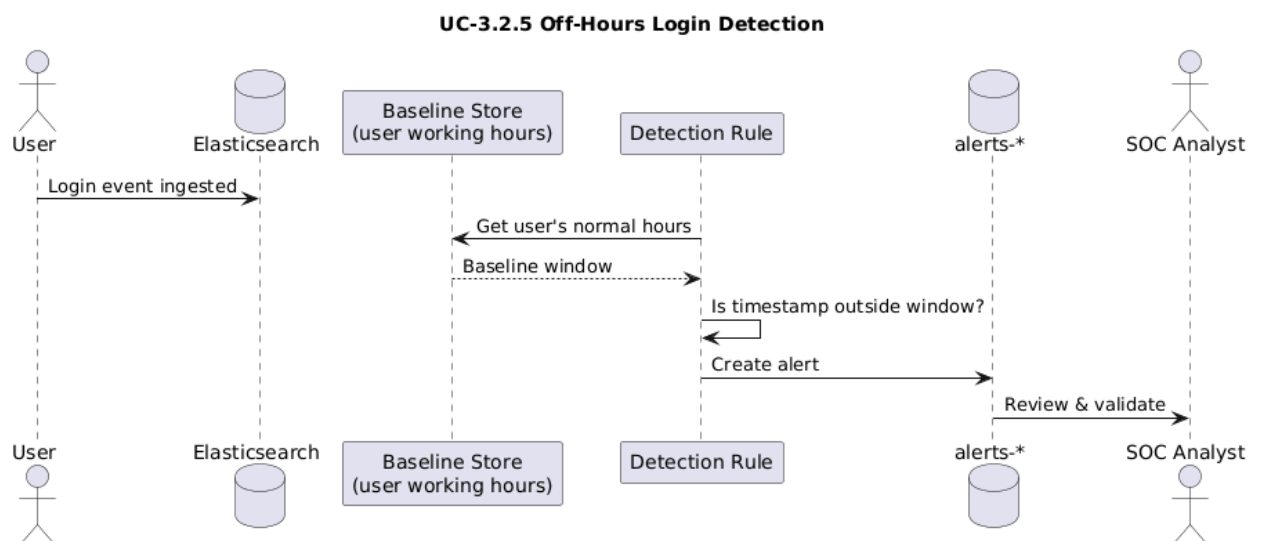
5.3 Detect Brute Force Attempts



5.4 Detect Impossible Travel

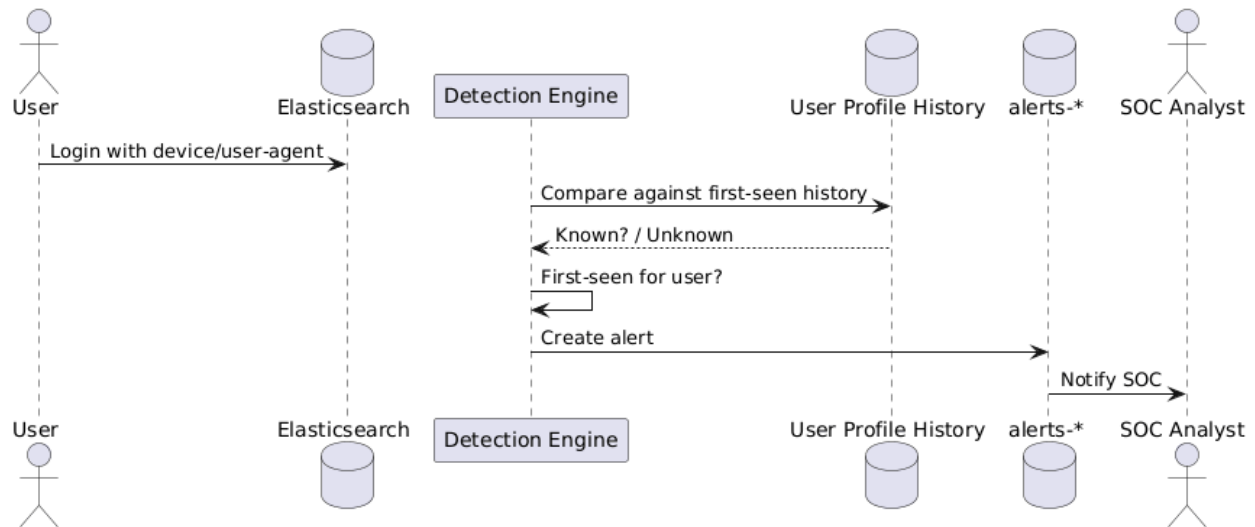


5.5 Off-Hours Login Detection



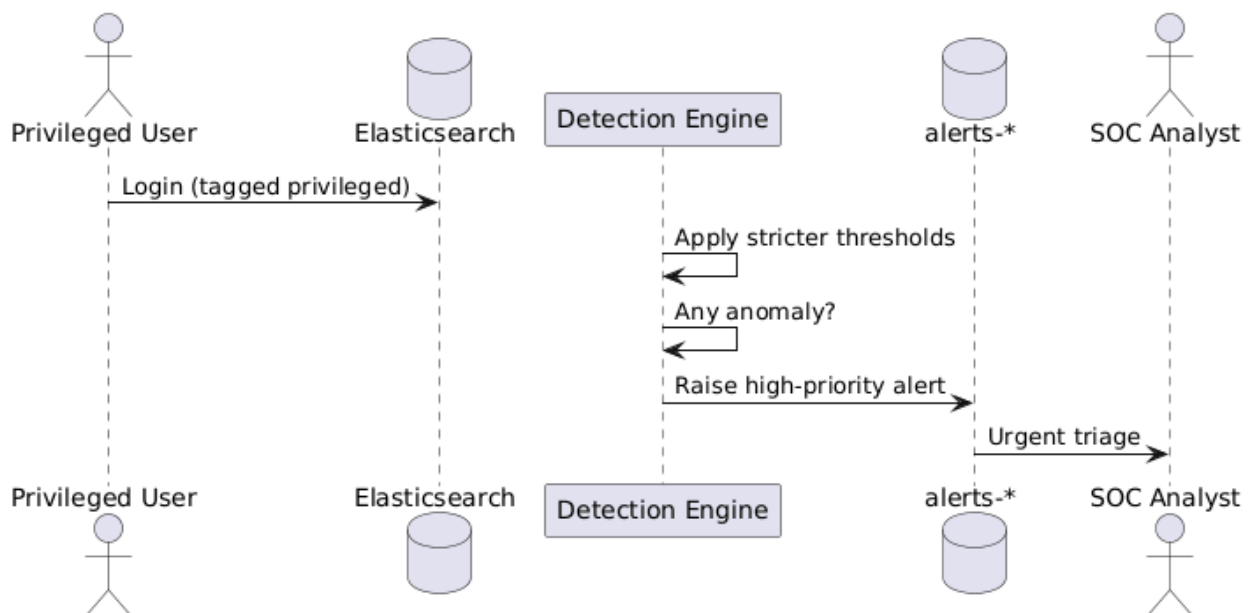
5.6 New Device/User-Agent Anomaly

UC-3.2.6 New Device / User-Agent Anomaly



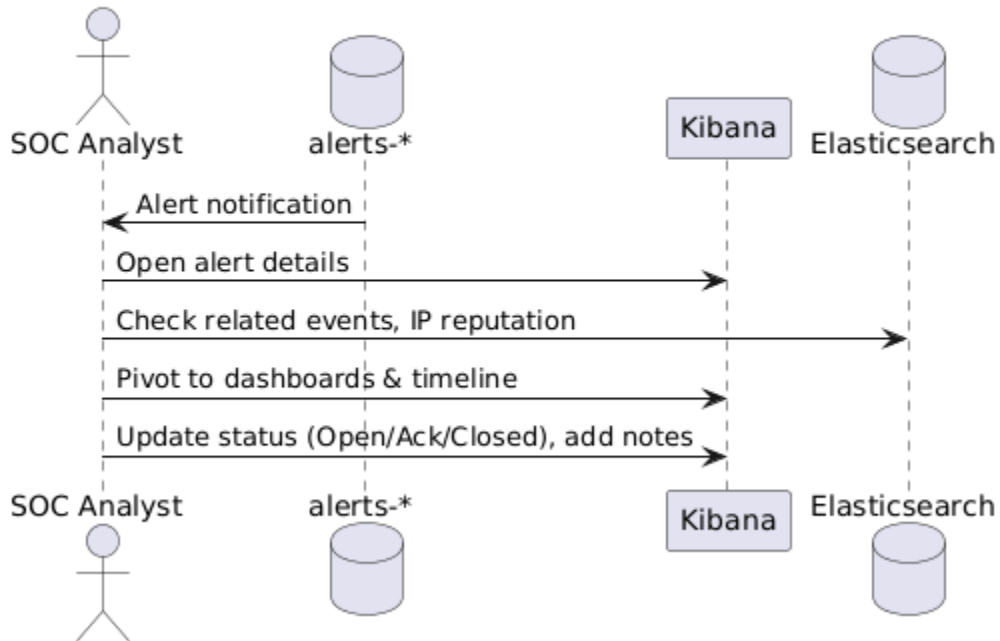
5.7 Privileged Account Watch

UC-3.2.7 Privileged Account Watch



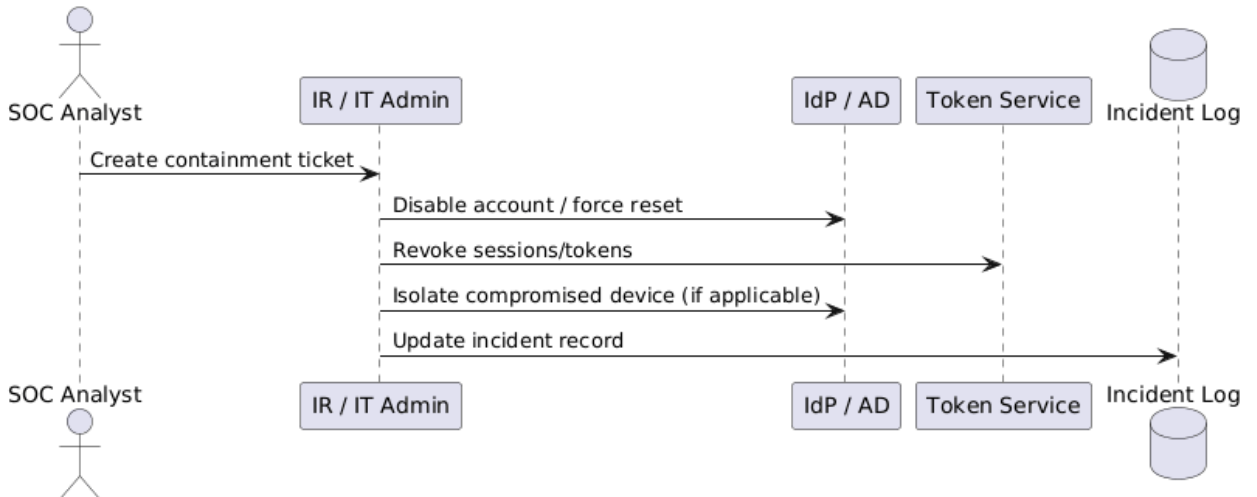
5.8 Triage Alert in Kibana

UC-3.2.8 Triage Alert in Kibana

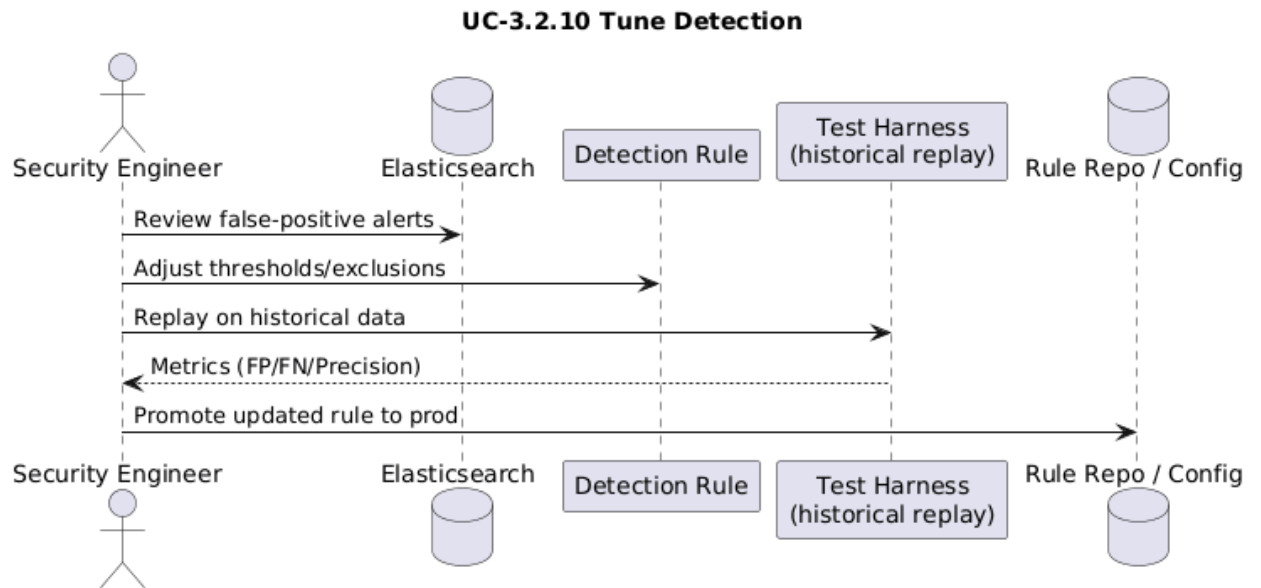


5.9 Contain Account

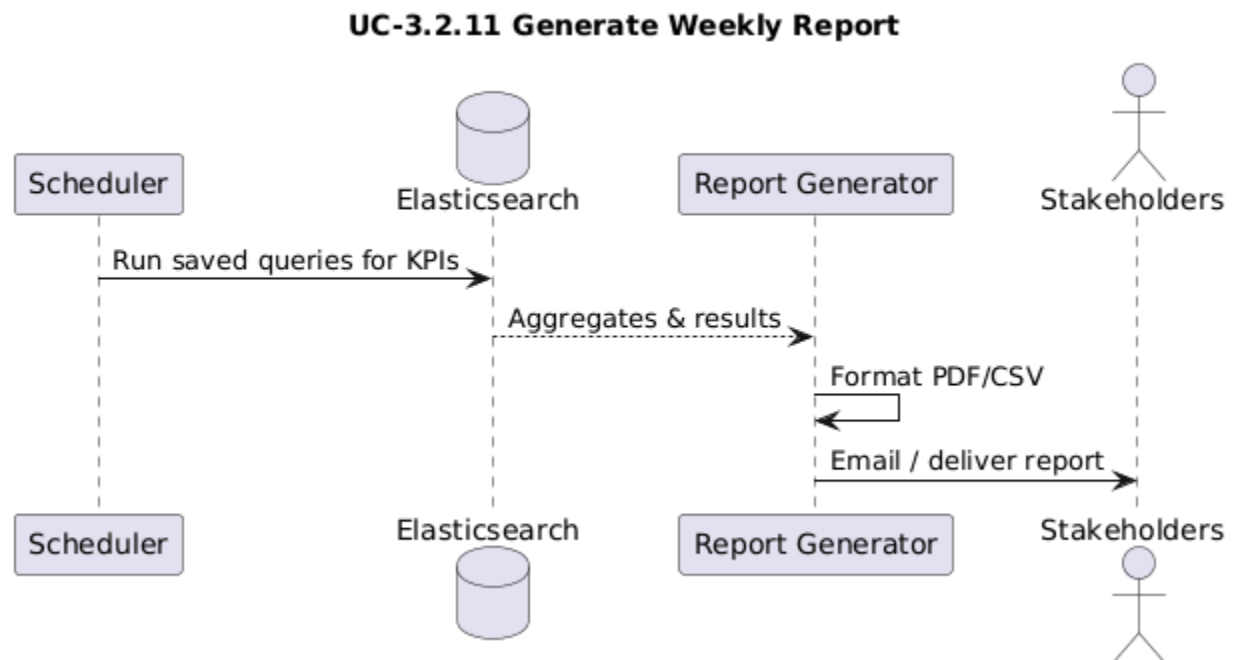
UC-3.2.9 Contain Account



5.10 Tune Detection

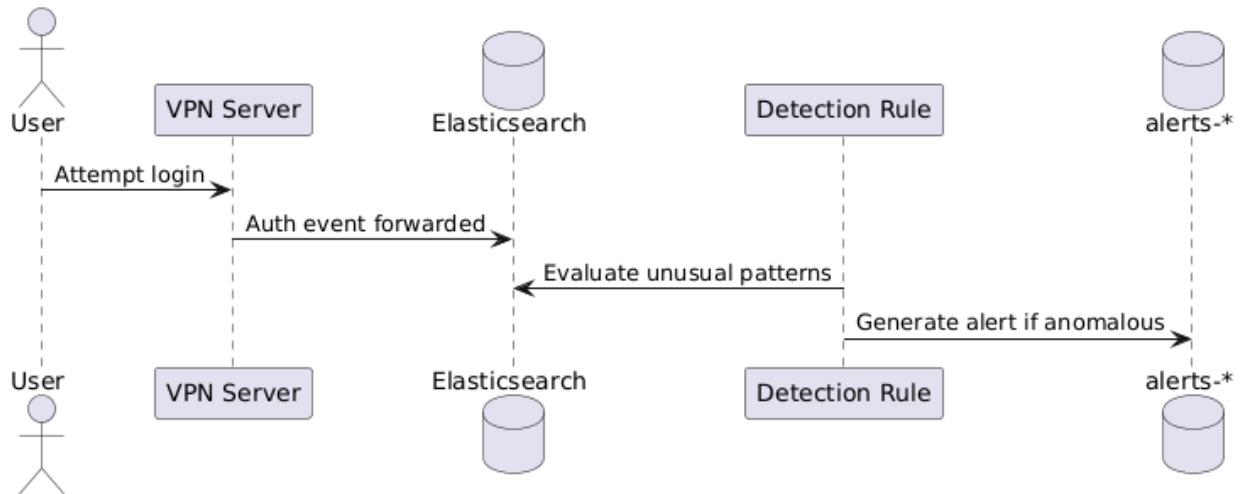


5.11 Generate Weekly Report



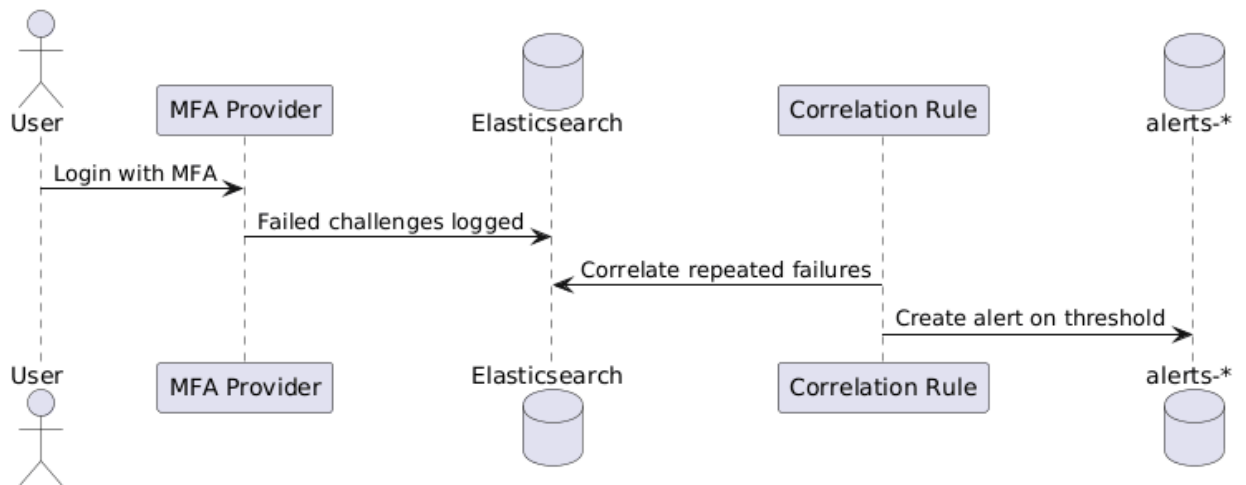
5.12 Monitor VPN Logins

UC-3.2.12 Monitor VPN Logins



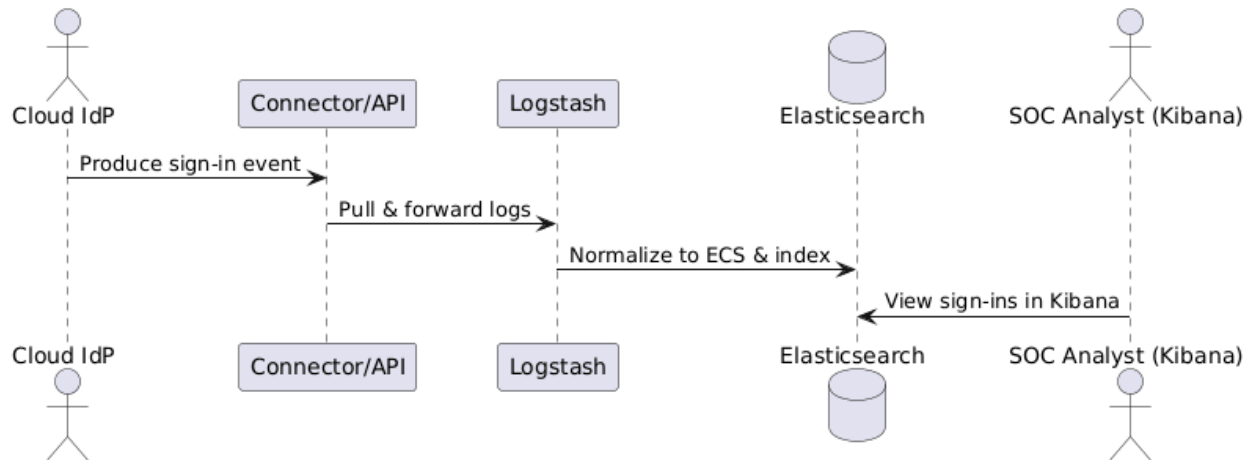
5.13 Detect MFA Failures

UC-3.2.13 Detect MFA Failures



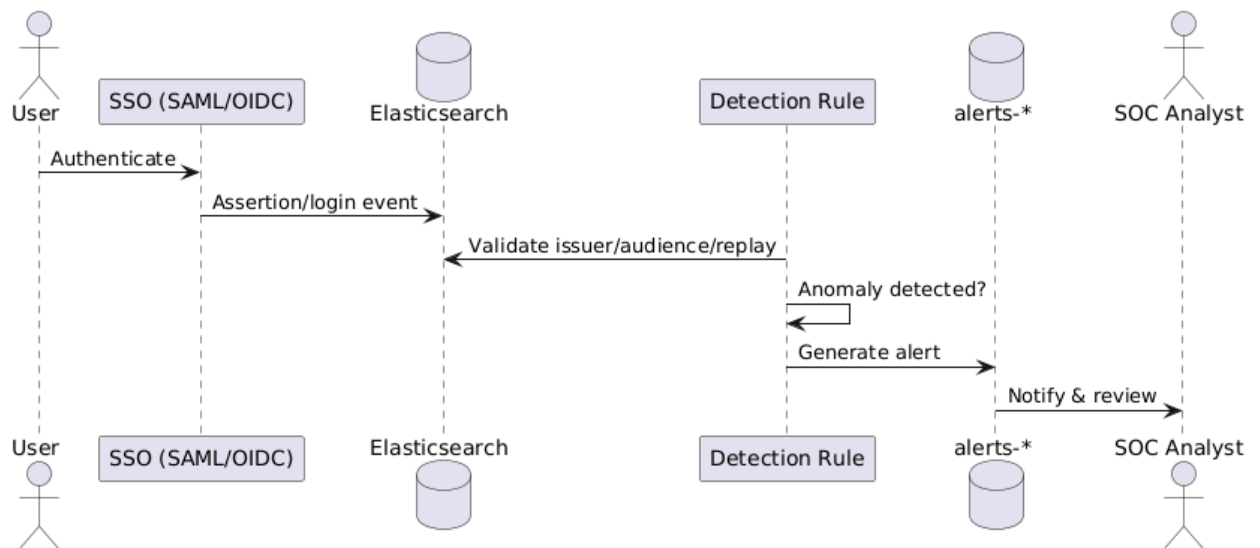
5.14 Ingest Cloud IdP Logins (AzureAD/Okta)

UC-3.2.14 Ingest Cloud IdP Logins (AzureAD/Okta)

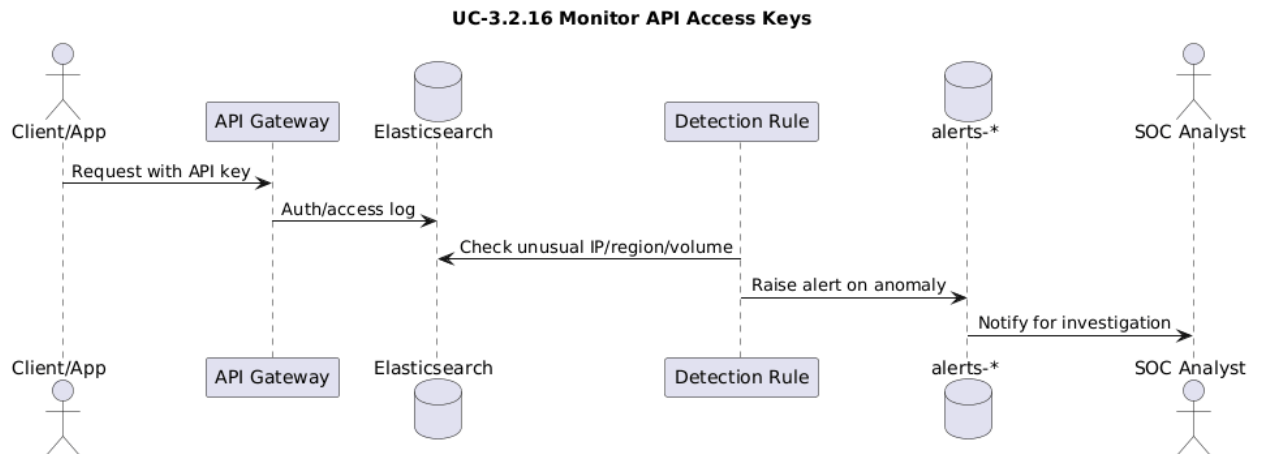


5.15 Detect SSO Anomalies

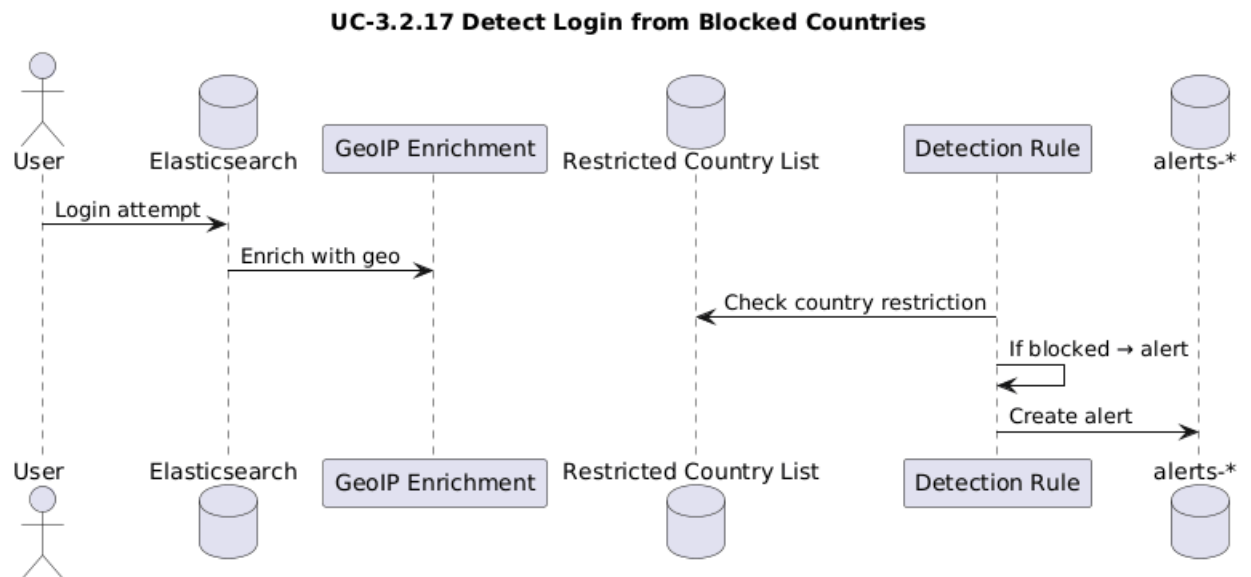
UC-3.2.15 Detect SSO Anomalies



5.16 Monitor API Access Keys

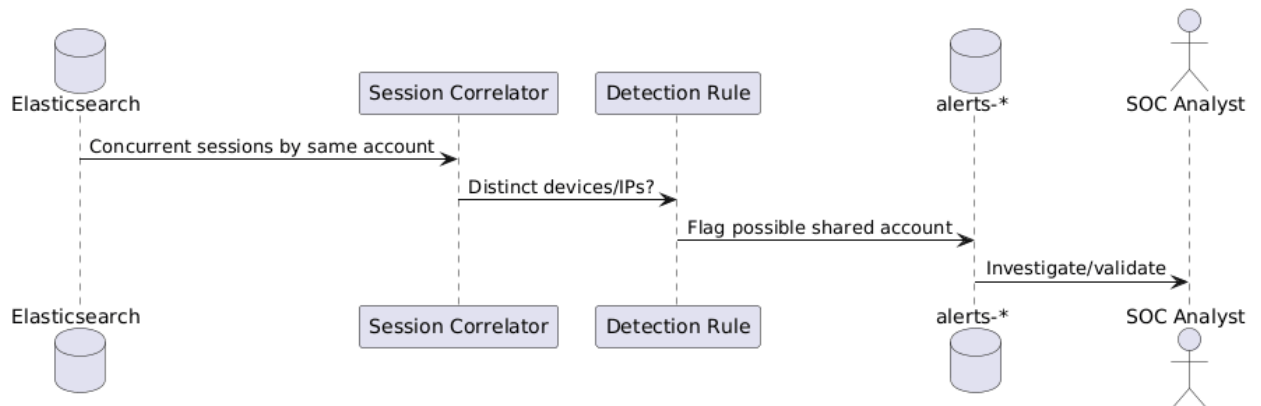


5.17 Detect Login from Blocked Countries



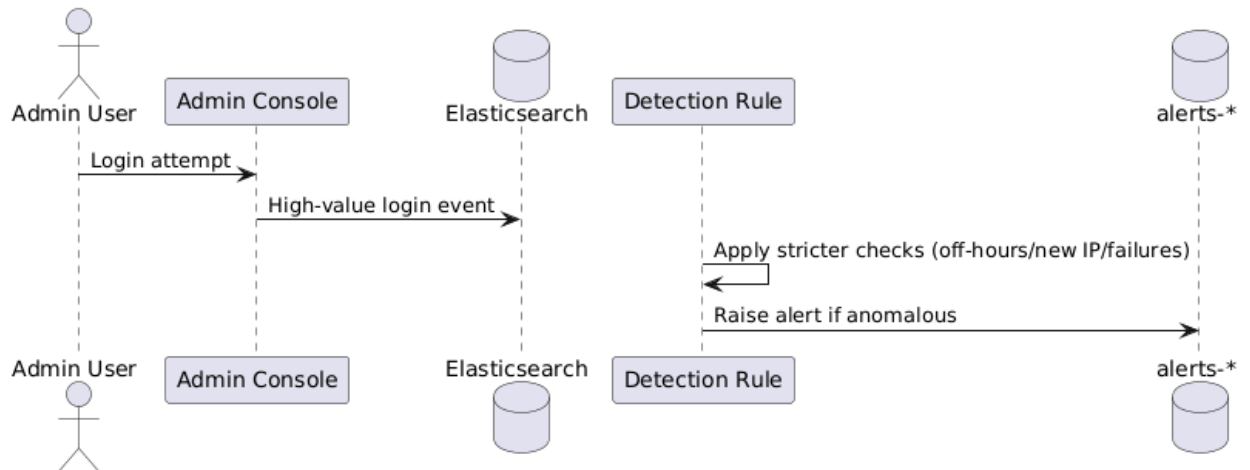
5.18 Detect Shared Account Usage

UC-3.2.18 Detect Shared Account Usage



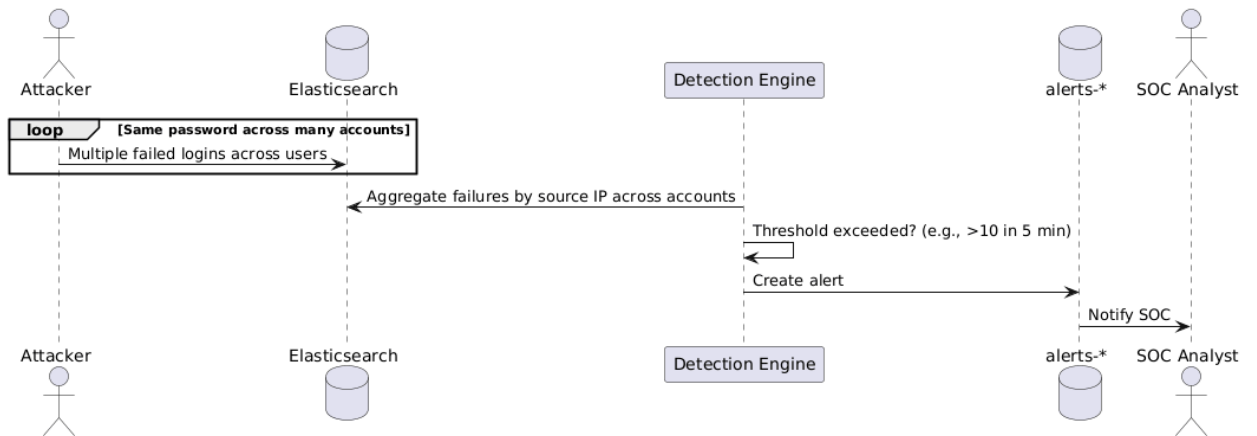
5.19 Monitor Administrative Console Logins

UC-3.2.19 Monitor Administrative Console Logins



5.20 Detect Password Spray Attack

UC-3.2.20 Detect Password Spray Attack



6. State Diagrams

6.1 Alert

States:

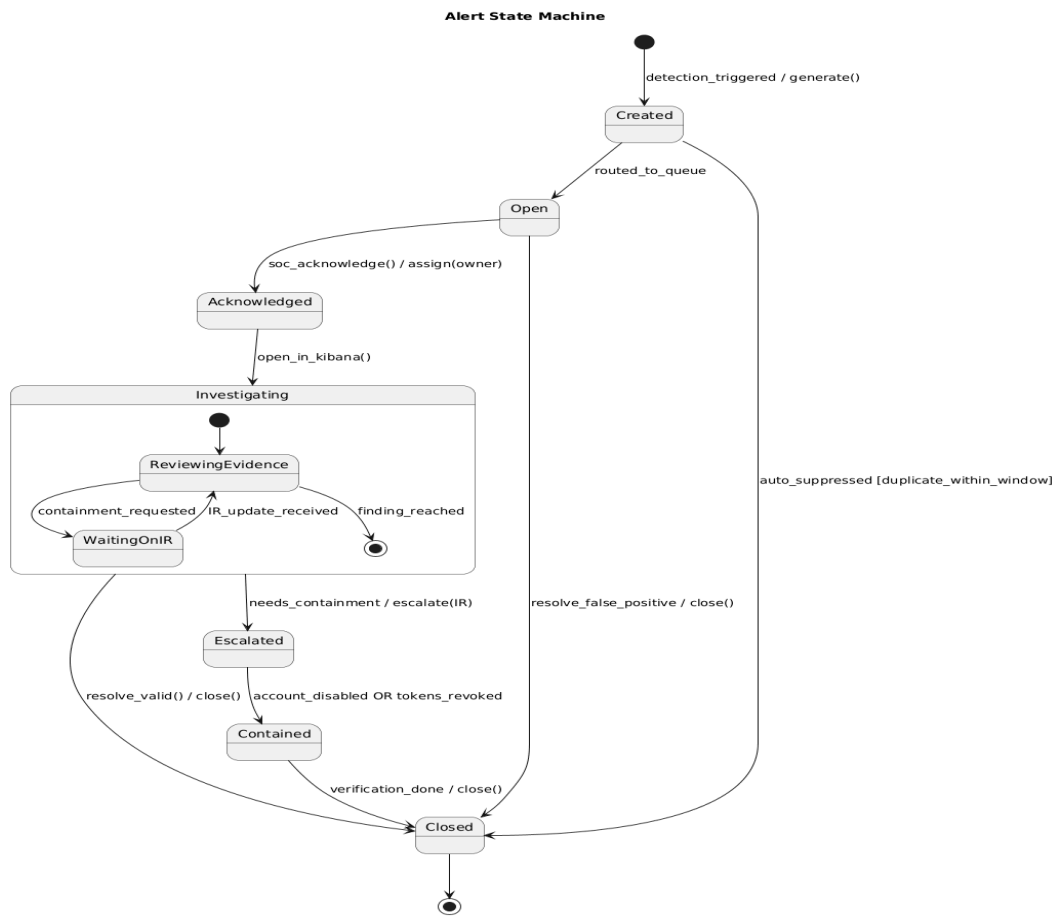
- Created → Alert object first generated by a detection rule.
- Open → Alert visible in system, waiting for analyst triage.
- Acknowledged → SOC analyst has taken ownership of alert.
- Investigating → Analyst gathering evidence and context.
- Escalated → Case handed off to higher-level IR team.
- Contained → Mitigation actions (disable account, revoke tokens) applied.
- Closed → Incident resolved, archived, and no longer active.

Key events:

- detection_triggered → New anomaly produces alert record.
- soc_acknowledge() → Analyst formally claims the alert.
- open_in_kibana() → Alert is loaded into Kibana for triage.
- escalate(IR) → Analyst escalates to incident response.
- close() → Alert lifecycle ended and archived.

Notes:

The Alert lifecycle models how a detection turns into a managed incident. It captures SOC ownership, triage, escalation, and eventual closure, ensuring accountability at every stage.



6.2 UserSession

States:

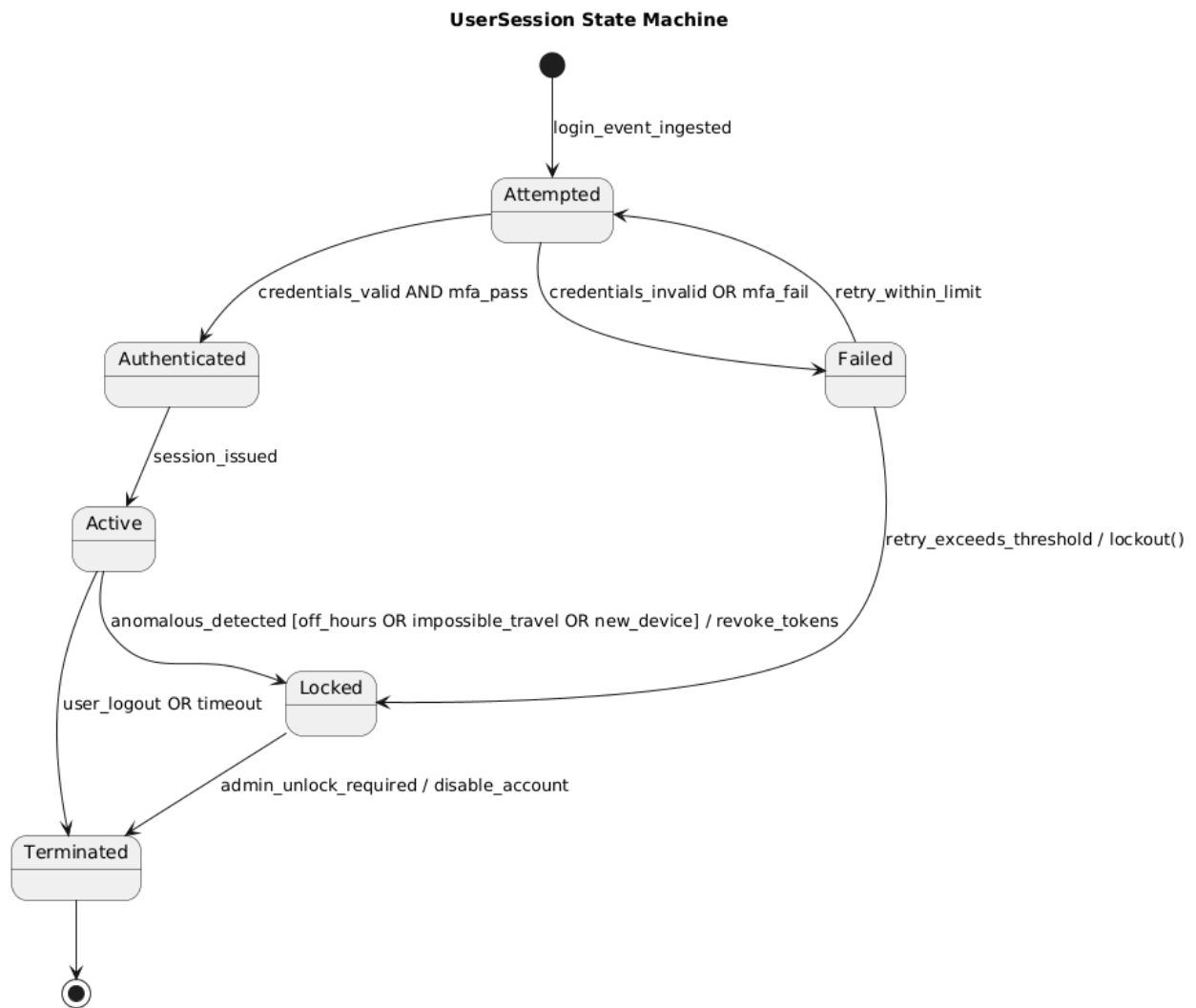
- Attempted → User submitted login credentials.
- Authenticated → Credentials accepted, MFA not yet checked.
- Active → Session established and running normally.
- Failed → Login denied due to bad credentials or MFA failure.
- Locked → Account locked after repeated anomalies or failures.
- Terminated → Session ended by logout, timeout, or admin action.

Key events:

- credentials_valid → Username/password match.
- mfa_pass → MFA verification successful.
- anomalous_detected → Suspicious behavior detected mid-session.
- timeout/logout → Session naturally ends or user exits.

Notes:

This diagram models the lifecycle of a user session, connecting logins, MFA, anomalies, and termination. It ties directly to anomaly detections like off-hours logins or impossible travel.



6.3 DetectionRule

States:

- Draft → Rule defined but not tested yet.
- Testing → Rule running in trial mode on historical or live data.
- Active → Rule fully deployed in production, generating alerts.
- Suppressed → Rule excluded temporarily (noise/maintenance).
- Retired → Rule permanently disabled or superseded.

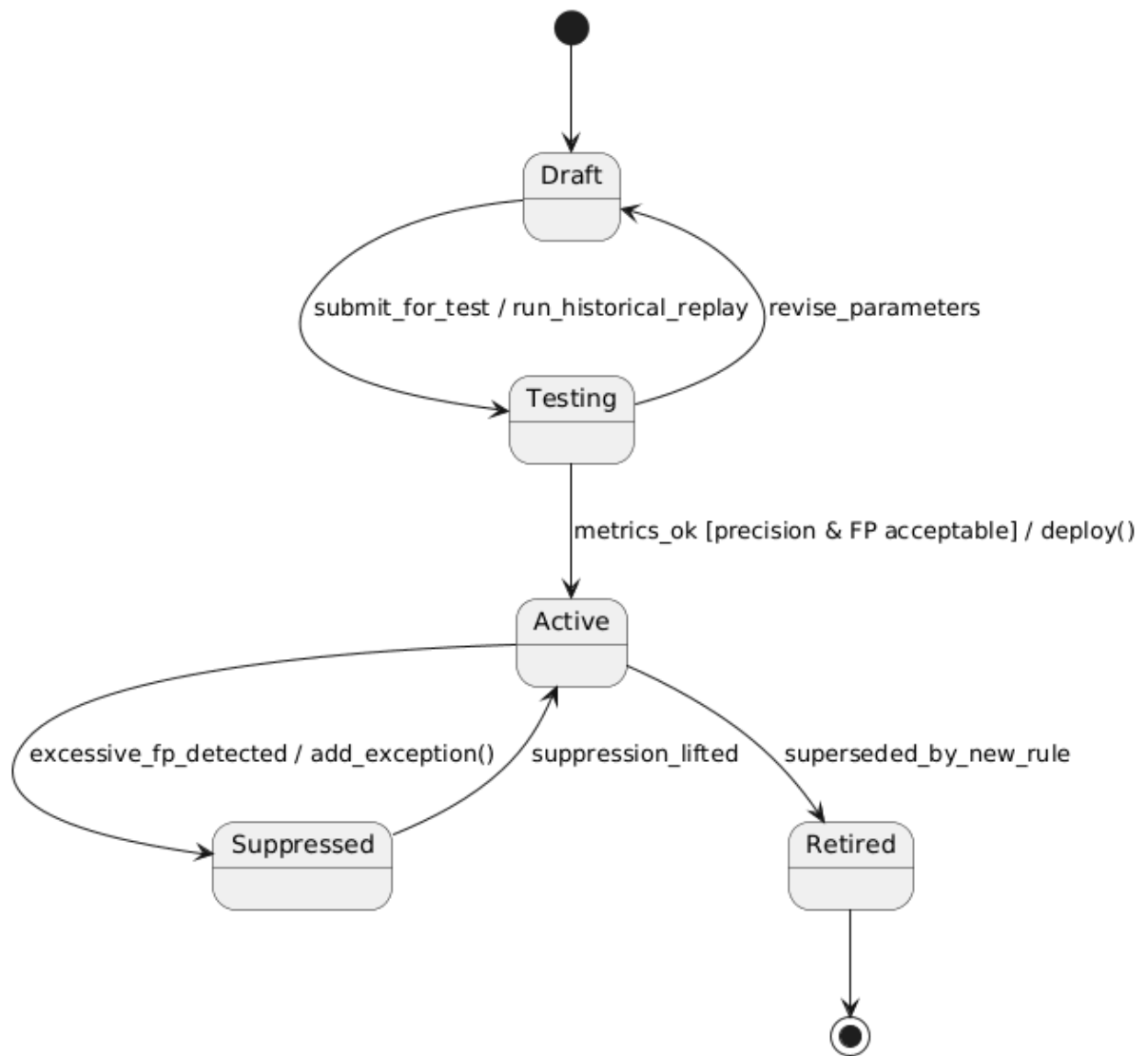
Key events:

- submit_for_test → Rule submitted for testing phase.
- deploy() → Rule deployed to production monitoring.
- add_exception() → Rule updated with exclusions/whitelist.
- supersede → Replaced by newer detection rule.

Notes:

The DetectionRule lifecycle mirrors how engineers define, validate, and refine anomaly checks. It captures promotion, suppression, and retirement to ensure effective threat coverage without excessive noise.

DetectionRule State Machine



6.4 LogEvent

States:

Received → Raw log received by ingestion system.

Parsed → Log successfully broken into fields.

Normalized → Data mapped into ECS schema for consistency.

Indexed → Event stored in Elasticsearch and searchable.

DeadLetter → Failed logs sent to error index for review.

Key events:

parse_ok → Parsing succeeded without error.

ecs_mapped → Data fields mapped into ECS standard.

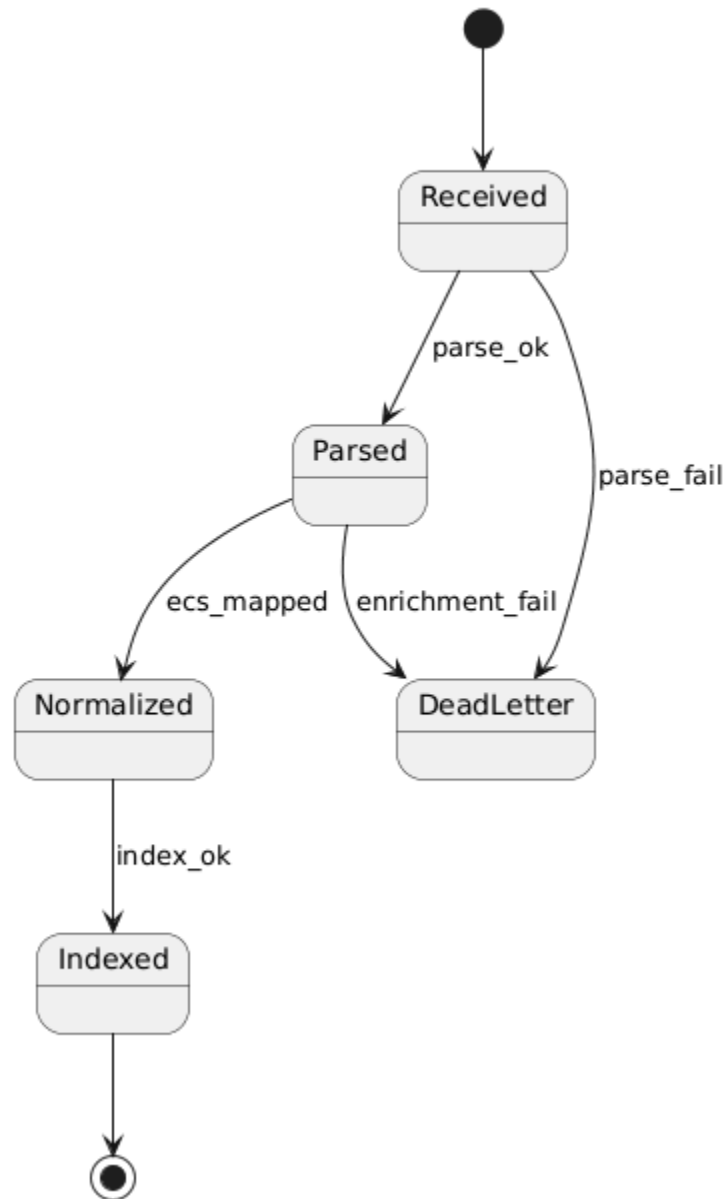
index_ok → Successfully written into index.

parse_fail → Parsing error encountered.

Notes:

This lifecycle models how raw authentication logs move through collection, parsing, normalization, indexing, and failure handling, ensuring visibility and error resilience in the pipeline.

LogEvent Processing State Machine



6.5 UserProfile (baselines)

States:

Empty → No history for this user yet.

Learning → System collecting initial login behavior.

BaselineReady → Stable profile built (devices, times, locations).

Stale → Profile outdated and requires refresh.

Key events:

observe_login() → New login event added to profile.

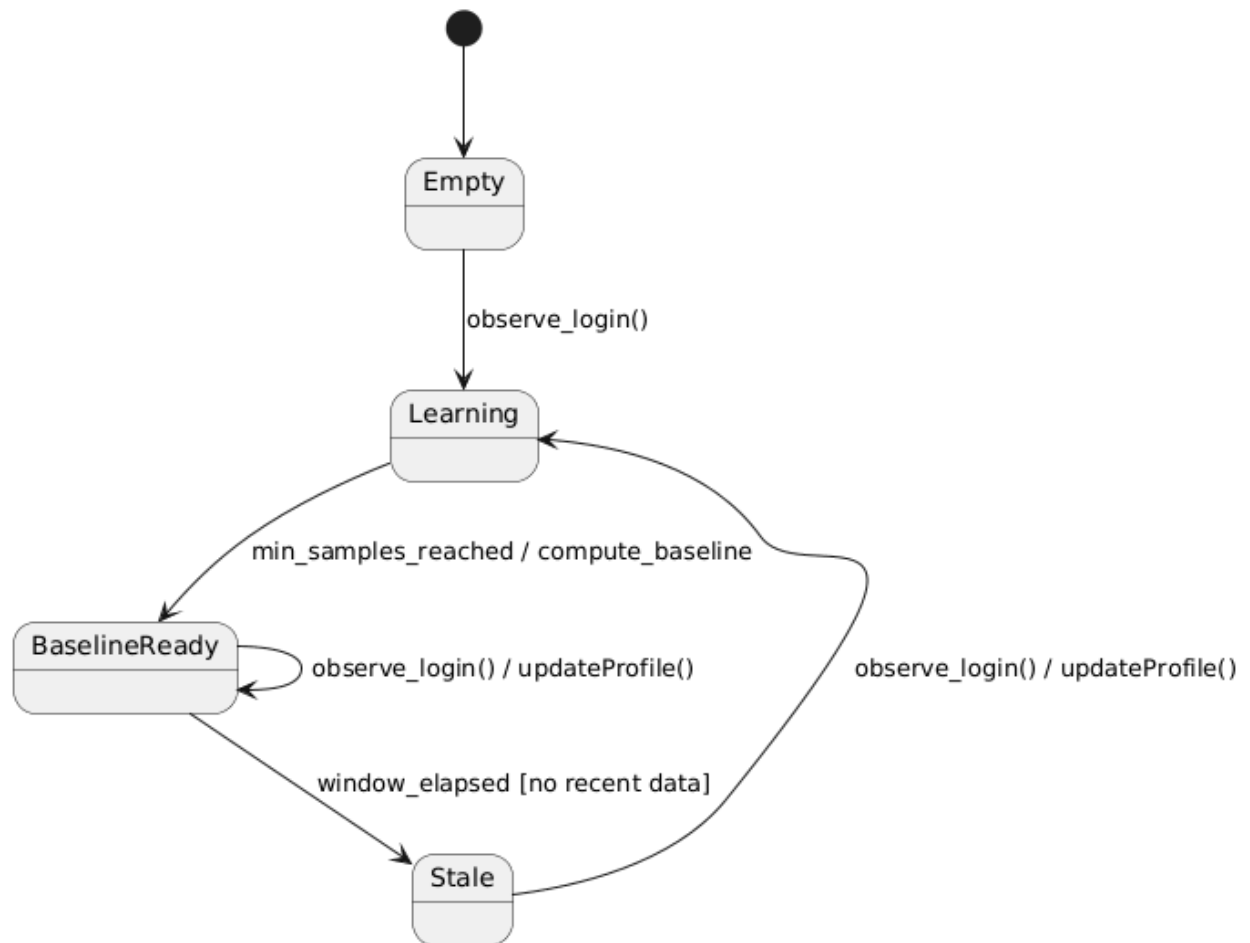
window_elapsed → Enough logins/time passed to establish baseline.

refresh_baseline → Profile updated with newer behavior.

Notes:

The UserProfile lifecycle supports behavioral analytics. It shifts from no data to stable baselines and refreshes when patterns change, enabling anomaly detections like new device or off-hours login.

UserProfile State Machine



6.6 ContainmentTicket (IR workflow)

States:

Open → Ticket created after confirmed compromise.

Assigned → Allocated to an incident responder.

InProgress → Mitigation actively underway.

Waiting → Paused due to pending dependency.

Resolved → Containment actions completed.

Blocked → Cannot progress due to external issue.

Closed → Fully wrapped up and archived.

Key events:

assign(ir) → Assigns ticket to IR staff.

disable_account → Account disabled/reset.

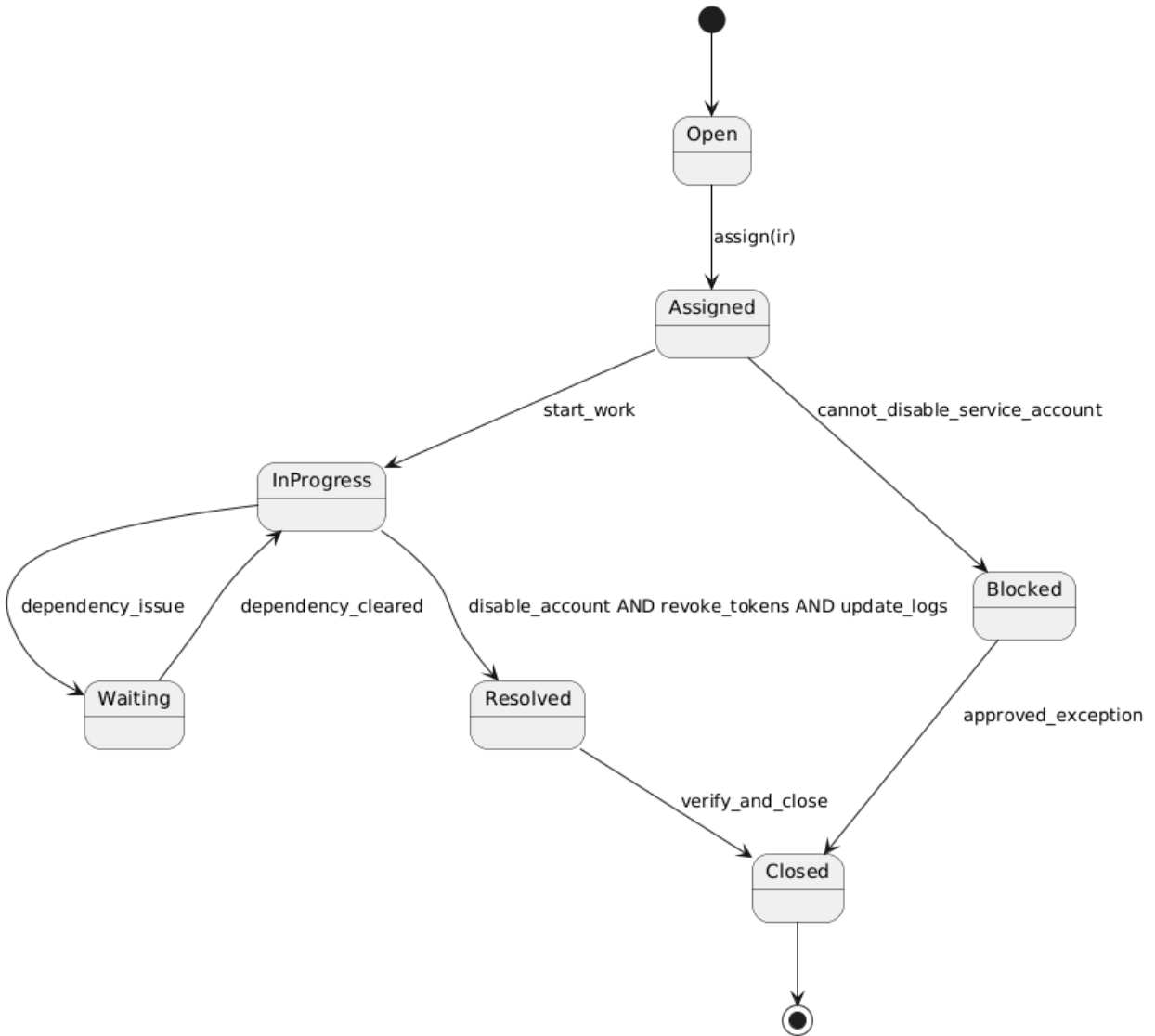
revoke_tokens → Tokens and sessions revoked.

dependency_issue → Ticket paused awaiting dependency.

Notes:

This lifecycle manages the containment process for compromised accounts. It reflects assignment, active mitigation, blockers, and closure, tracking accountability in IR workflows.

ContainmentTicket State Machine



6.7 AdminConsoleSession

States:

- Attempted → Admin console login initiated.
- Authenticated → Credentials accepted.
- Elevated → Admin privileges escalated successfully.
- Flagged → Activity flagged as suspicious (IP/time anomaly).
- Ended → Session closed normally or forcibly.

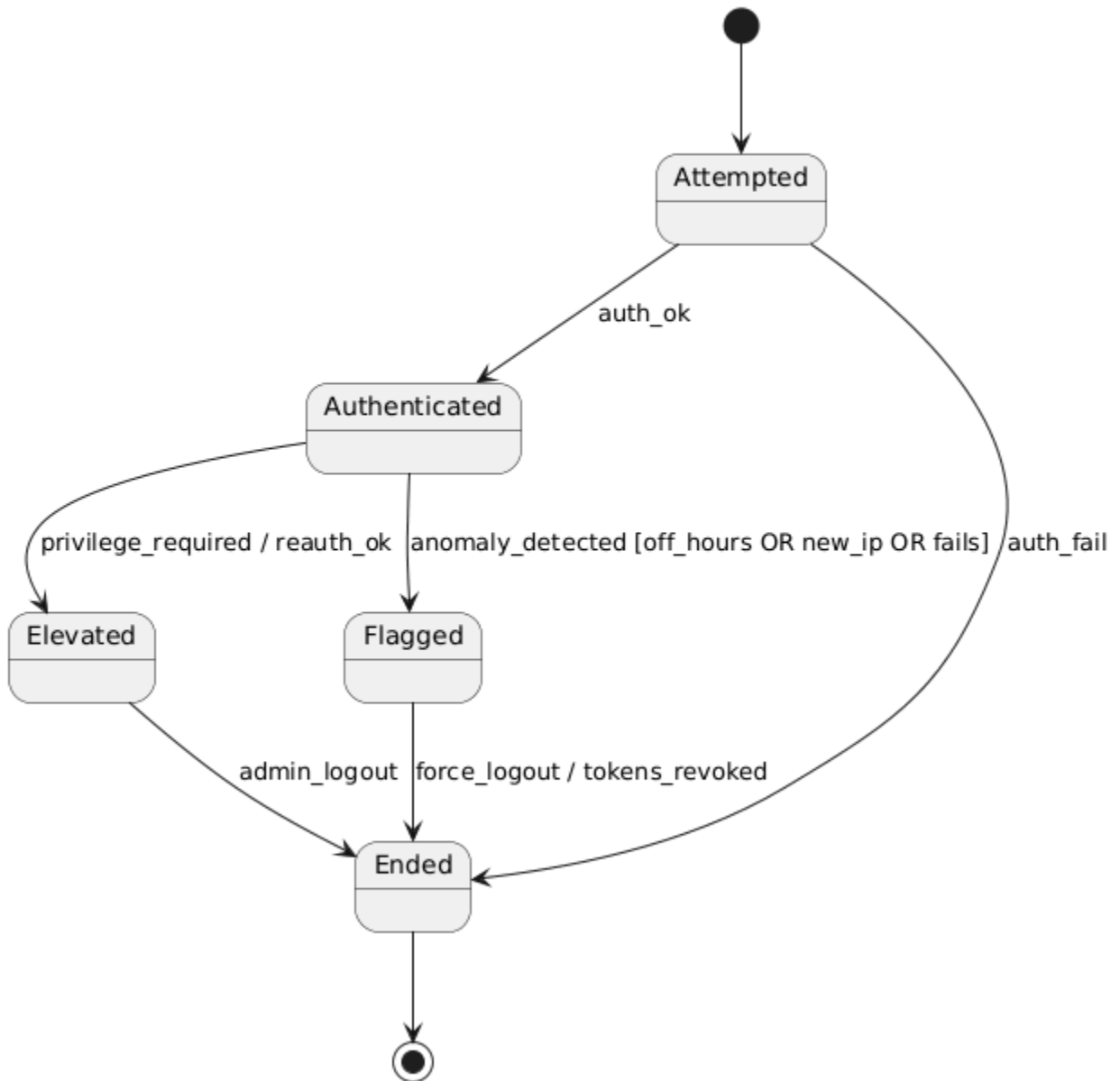
Key events:

- privilege_required → Request for elevated privileges.
- new_ip_offhours → Login from unusual IP or off-hours.
- logout → Admin session terminated.

Notes:

This diagram models high-value admin console logins with stricter anomaly checks, covering privilege elevation, suspicious conditions, and session closure.

AdminConsoleSession State Machine



6.8 APIKey

States:

- Issued → New API key generated.
- Active → Key in valid, ongoing use.
- Flagged → Suspicious behavior associated with key.
- Revoked → Key permanently disabled.

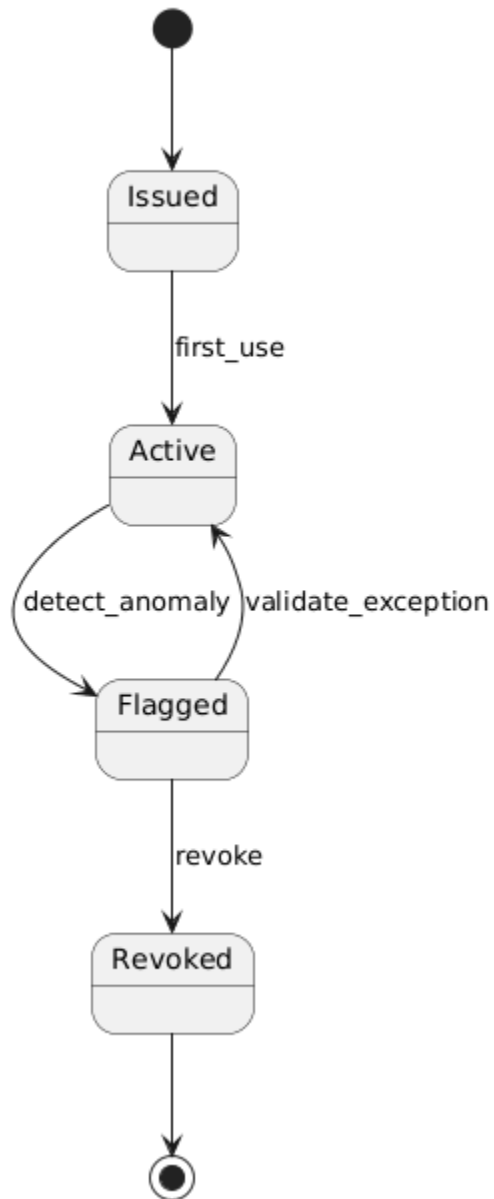
Key events:

- detect_anomaly → Unusual key usage detected.
- validate_exception → Mark flagged activity as valid.
- revoke → Key revoked by admin/SOC.

Notes:

The APIKey lifecycle highlights the security risks of credentials, capturing their issuance, active use, anomaly detection, and eventual revocation when compromised.

APIKey State Machine



6.9 Dashboard (Investigation workspace)

States:

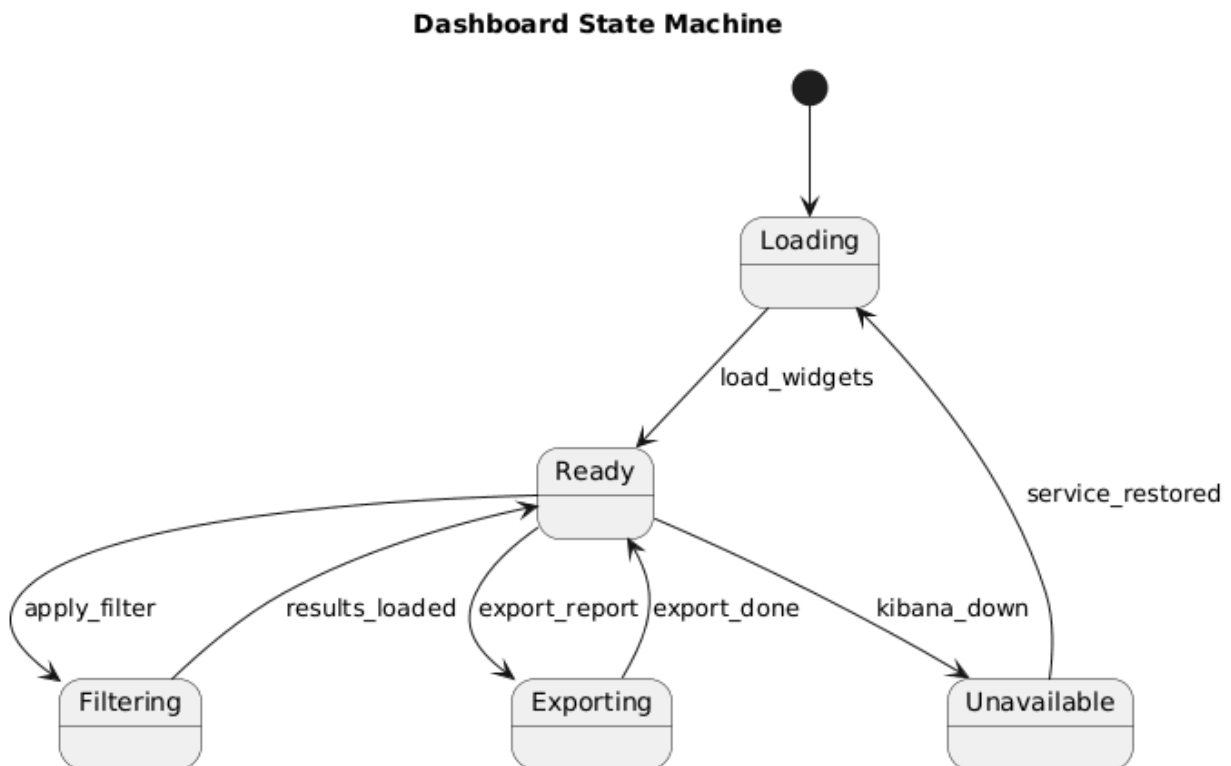
- Loading → Widgets and data currently loading.
- Ready → Dashboard interactive and available.
- Filtering → Filters applied to refine view.
- Exporting → Report generation in progress.
- Unavailable → Dashboard inaccessible (e.g., Kibana down).

Key events:

- load_widgets → Load dashboard components.
- apply_filter → Apply user-defined filter.
- export_report → Export dashboards as report.
- kibana_down → Dashboard unavailable due to outage.

Notes:

This lifecycle reflects how dashboards move from loading to ready state, support interactive filtering and exporting, and handle downtime gracefully.



6.10 IdPConnector

States:

- Idle → Connector waiting between polls.
- Polling → Fetching identity provider logs.
- Transforming → Logs normalized into ECS format.
- Pushing → Events being forwarded to Elasticsearch.
- Error → Error occurred during log handling.
- Disabled → Connector turned off by admin.

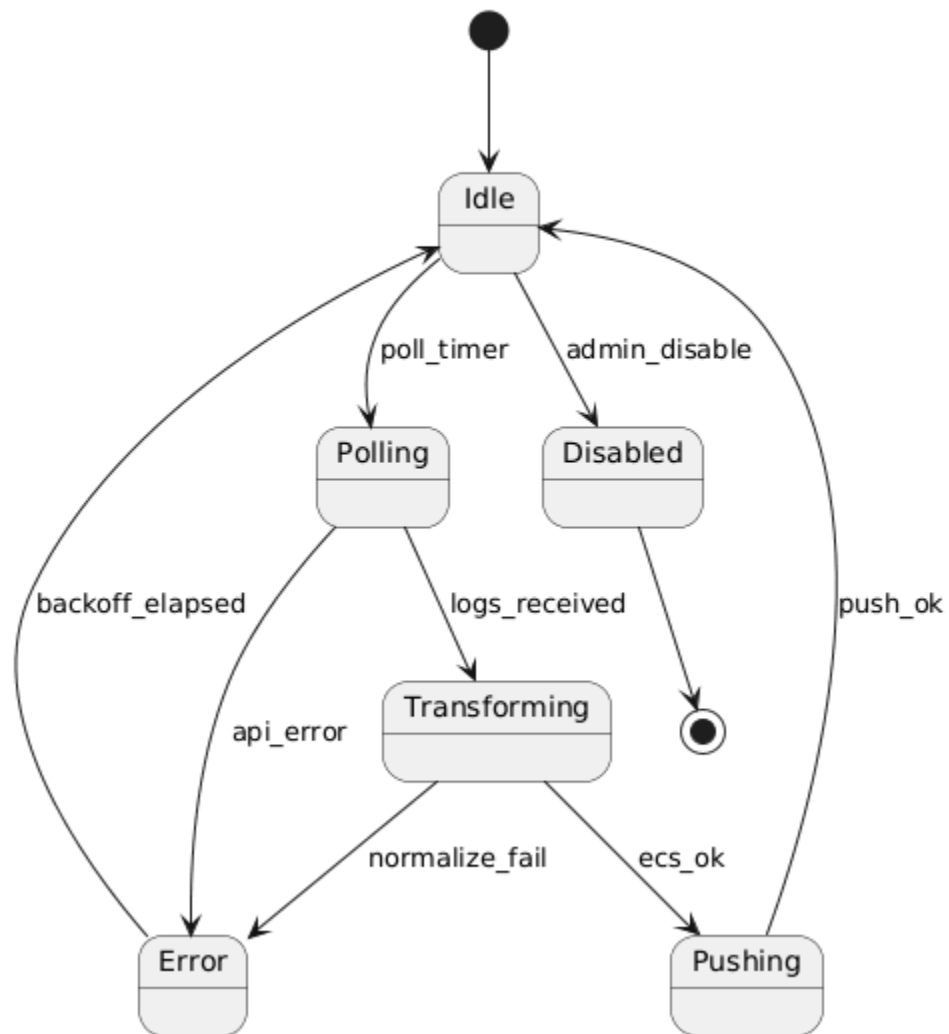
Key events:

- poll_timer → Poll triggered.
- parse_ok → Logs parsed successfully.
- ecs_ok → Normalization succeeded.
- push_ok → Data pushed to index.
- api_error → API failure encountered.

Notes:

This lifecycle shows the flow of authentication data from cloud IdPs into ELK. It captures collection, transformation, delivery, and resilience against errors or outages.

IdPConnector State Machine



6.11 IngestionAgent (Wazuh/Filebeat on host)

States:

- Running → Actively collecting and sending logs.
- Buffering → Temporarily storing logs due to outage.
- Sending → Forwarding buffered logs.
- Backoff → Waiting before retry after failure.
- Stopped → Agent shut down.

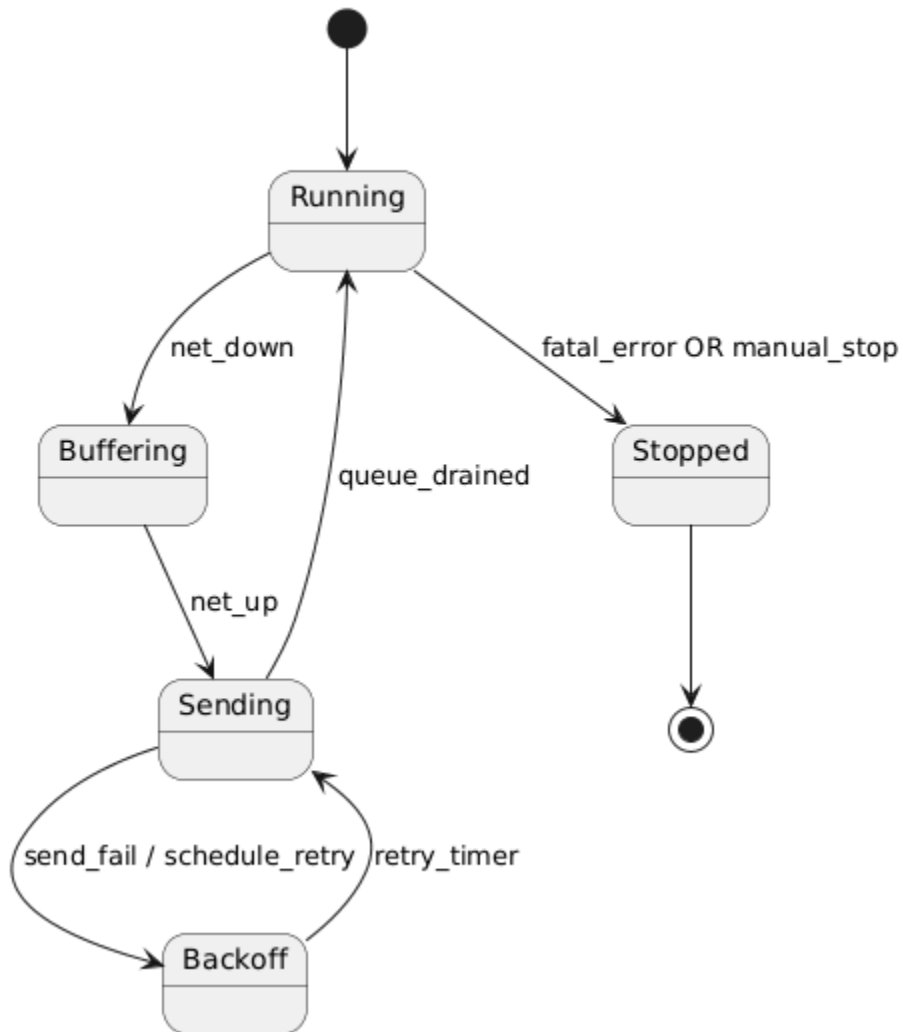
Key events:

- net_down → Network outage detected.
- net_up → Network restored.
- queue_full → Buffer at capacity.
- fatal_error → Irrecoverable error stops agent.

Notes:

This diagram captures agent behavior under normal operations, outages, and retries. It ensures logs are buffered and eventually delivered without data loss.

IngestionAgent State Machine



6.12 VPNConnection

States:

- Requesting → VPN session request sent.
- Authenticating → Credentials being validated.
- Established → VPN session active.
- Dropped → Connection terminated unexpectedly.
- Blocked → Connection denied due to policy/anomaly.

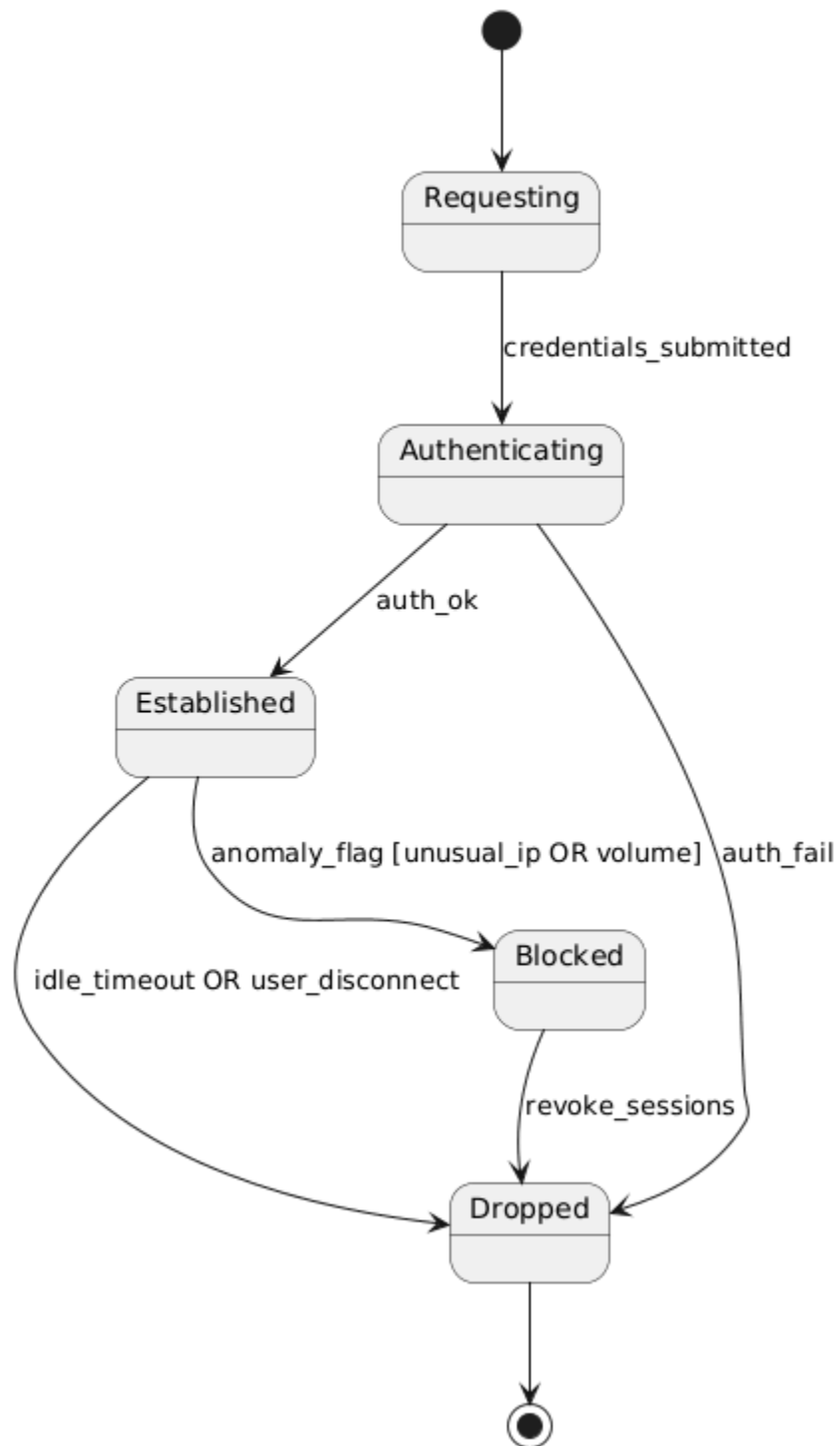
Key events:

- auth_ok → Authentication successful.
- auth_fail → Authentication failed.
- anomaly_flag → Suspicious VPN activity detected.
- idle_timeout → VPN disconnected after inactivity.

Notes:

The VPNConnection lifecycle reflects secure remote access, tracking normal session establishment and abnormal terminations caused by anomalies or policy enforcement.

VPNConnection State Machine



6.13 SecurityEngineer

States:

ReviewingAlerts → Checking false positives in alerts.
DraftingRule → Writing new anomaly detection rule.
TestingRule → Rule in test mode with limited data.
ActiveRule → Rule deployed to production.
Monitoring → Watching rule performance in real-time.
Tuning → Adjusting rule thresholds/exceptions.
Suppressed → Temporarily disabling noisy rule.
Retired → Rule deprecated and archived.

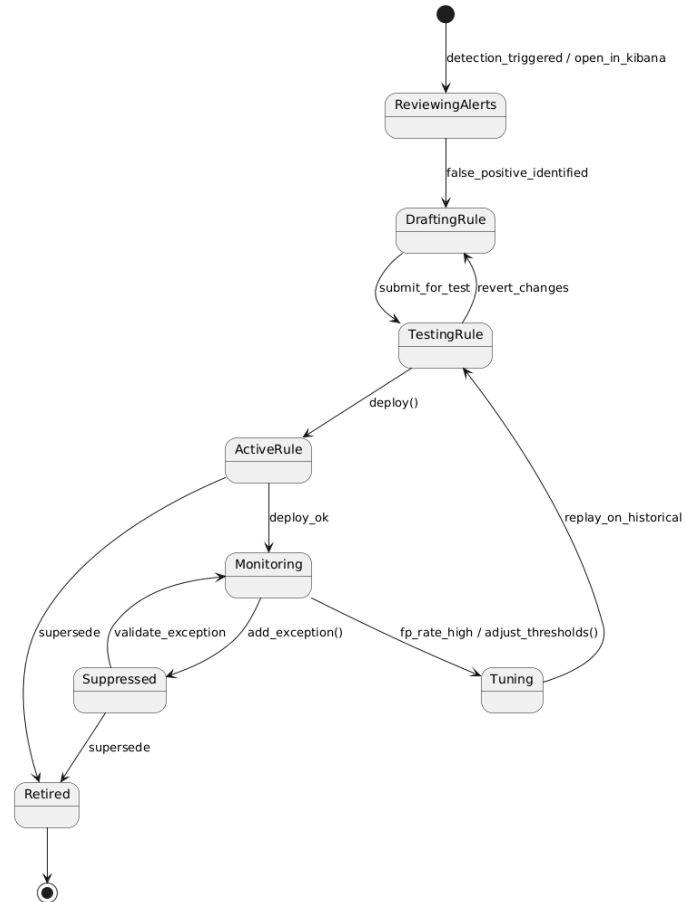
Key events:

false_positive_identified → Alert judged as false positive.
submit_for_test → Move rule to testing.
deploy() → Push rule to production.
revert_changes → Roll back faulty rule update.
add_exception() → Add whitelist/exclusion.
supersede → Rule replaced by improved version.

Notes:

This lifecycle models the workflow of a security engineer. It links false positive review, drafting and testing rules, deployment, monitoring, tuning, and retirement of rules.

SecurityEngineer State Diagram



7. Data Requirements

< If applicable, define the data needed for training, validation, and testing.>

Data Sources	● System Logs: Authentication Logs ● Test data: Simulated suspicious login attempts in a controlled environment.
Data Requirements	● The data must include a mix of successful logins and failed logins. ● The failed logins should include all the different types of anomalies. ● Data should include logs from multiple platforms (windows, mac, linux). ● Less than 5% of the entries should be corrupted. ● Data must also include edge cases.;;'p'['
Model Requirements	● For future training of a machine learning model, train a simple anomalous login detection model on login data. ● Retrain the model after a set period of time on newer data to ensure the detection system is kept up to date. ● The target of this model would be a minimum of 90%.

8. Non-functional Requirements / Quality Attributes

<Requirements must be testable>

Sr#	Requirements
1	The system should not utilize more than 1 GB of memory at any time during its execution.
2	The system should not fail more than 3 times every 24 hours; if it does, it should recover within 5 minutes.

3	The system should be able to process at least 10,000 log entries per second without performance degradation.
4	The alerting mechanism should deliver notifications within 30 seconds of anomaly detection.
5	System dashboards must be accessible only to authorized users.
6	The system should allow simultaneous access by at least 5 users without performance degradation.
7	The system's dashboard must refresh with new data every 10 seconds to ensure real-time data is visible.
8	The system should undergo daily health checks with results logged for administrators.

9. Security Requirements

< Go through OWASP top 10 security risks in the following categories:

- a. OWASP Top Ten: <https://owasp.org/www-project-top-ten/>
- b. OWASP Mobile Top 10: <https://owasp.org/www-project-mobile-top-10/>
- c. OWASP Machine Learning Security Top Ten: <https://owasp.org/www-project-machine-learning-security-top-10/>
- d. OWASP Top 10 API Security Risks: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

- (a) Select **security risks** that you think are primary threats for your system. While doing this, carefully consider the information/functionality that is most vulnerable from security perspective in the context of your project.
- (b) For each security risk (identified above), identify **potential losses** (e.g., financial loss, total business loss, litigation etc.) if you do not take necessary measures to address the identified security risks.
- (c) Identify the **controls** (e.g., input validation, audit logs, multi-factor authentication, user roles etc.) that should be implemented in your system to address the identified security risks.

Sr #	Security Risks	Potential Losses	Controls
1	Broken Access Control	Sensitive user login data is exposed.	Only security engineers will have update rights.
2	Input Manipulation Attack	Repeated attempts can cause the system to ignore the real threats.	Simulate the fake inputs to ensure the system's accuracy remains unaffected.
3	Data Poisoning Attack	Can cause the system's detection quality to drop over time.	Pre-process the data.
4	Using outdated versions of client-side and/or server-side components	Exploitation of known vulnerabilities in the older versions.	Obtain components from their official links

5	Hardcoding credentials	Credentials are exposed so hackers may be able to gain access	A security testing process would take place in order to ensure credentials are not exposed in such ways.
---	------------------------	---	--

10. Security Engineer

< Each team must designate one member as the **Security Engineer**. While the entire team is responsible for implementation of the project's security features, the Security Engineer will take the lead in overseeing and ensuring the overall security of the project. >

Name of the Security Engineer	Muhammad Aaffan Khan Niazi
--------------------------------------	----------------------------

11. Use of Generative AI

<Mention here how generative AI was used in preparation of this artifact.>

12. Who Did What?

Name of the Team Member	Tasks done
Aaffan Niazi	Use Cases, description of 20 use cases.
Mohammad Mustafa	Security requirements, non-functional requirements, data requirements.
Mustafa Hussain	Introduction, System Actors, Use cases diagrams, Class diagram and descriptions
Shehroz Faryad	Sequence Diagrams, State Diagrams and Descriptions

13. Review checklist

Before submission of this deliverable, the team must perform an internal review. Each team member will review one or more sections of the deliverable.

Section Title	Reviewer Name(s)
2,3,4,5	Muhammad Aaffan Khan Niazi
6,7,8	Mustafa Hussain
9,10	Mohammad Mustafa
3, 4, 5, 6	Shehroz Faryad